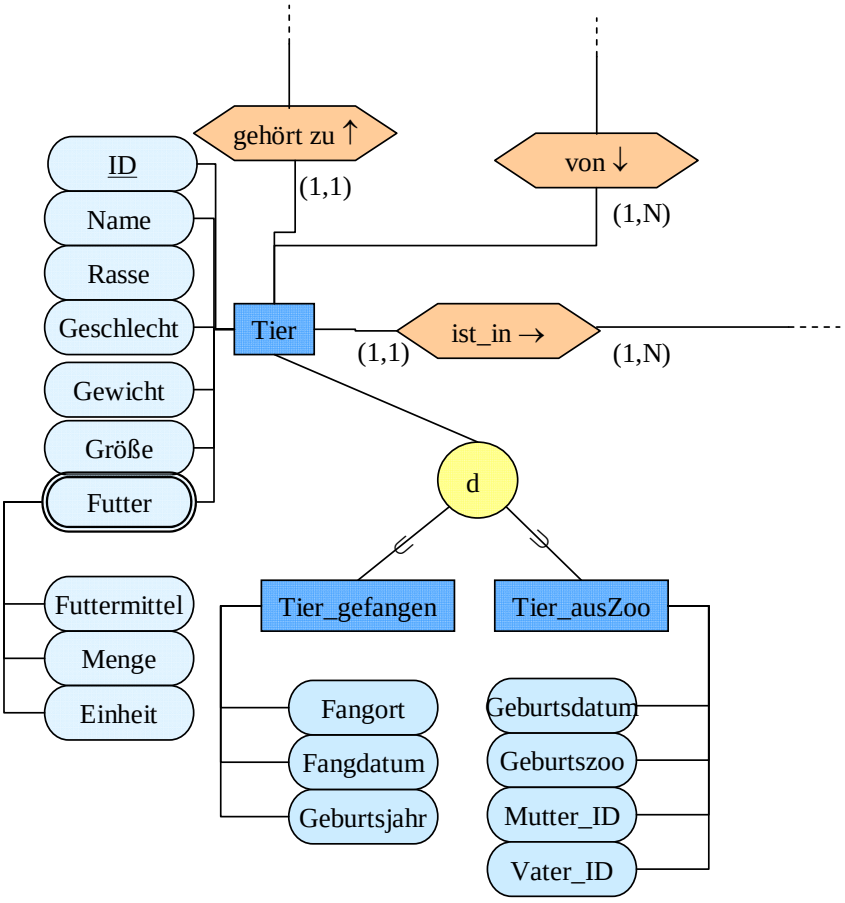
Datenbanksysteme



© Prof. Dr. Martin Hulin
Hochschule Ravensburg-Weingarten
Technik | Wirtschaft | Sozialwesen
Fakultät Elektrotechnik und Informatik
Tel.: 0751 552515 oder 0751 501 9733
E-Mail: hulin@hs-weingarten.de
Web: http://portal.hs-weingarten.de/web/hulin
Raum: T011

06.10.2011

Literaturempfehlungen

1. Ramez Elmasri, Shamkant B. Navathe: "Fundamentals of Database Systems", 5. Auflage, Verlag Benjamin/Cummings, Redwood City, 2007, ISBN 9780321369574
   oder deutsche Übersetzung "Grundlagen von Datenbanksystemen", 3. Aufl. 2002, ISBN 3-8237-7021-3
   Standardlehrbuch, Überblick über die gängigen und neue Datenbankmodelle, ausführlich, viele Beispiele. 10 Exemplare in der Bibliothek vorhanden.
   Bachelorausgabe 3. Auflage, 2009, ISBN 978-3-8689-4012-1, Preis 29,95 €, siehe http://www.pearson-studium.de/main/main.asp?page=bookdetails&isbn=9783868940121

2. Hald, Anton: "Datenbank-Engineering für Wirtschaftsinformatiker", Vieweg Verlag, 1995, ISBN 3-528-05436-0
   10 Exemplare in der Bibliothek vorhanden
   Ein guter Überblick über moderne Datenbank-Systeme; passt gut zur Vorlesung; deutsch.

3. J. Hoffer, M. Prescott, F. McFadden: "Modern Database Management", Pearson Education, 2005, ISBN 0-13-127388-4
   Standardlehrbuch, Alternative zu Buch Nr. 1 (Elmasri, Navathe), etwas moderner aufgemacht, dafür nur in Englisch

4. Uwe Hohenstein, Volkmar Pleßler: "Oracle9i – Effiziente Anwendungsentwicklung mit objektrelationalen Konzepten", 2. Auflage, DPunkt-Verlag, 2002, ISBN 3-89864-125-2
   Speziell für embedded SQL, aber auch SQL

5. RRZN/Universität Hannover (Hrsg.): "SQL Grundlagen und Datenbankdesign", 4. Auflage.
   Bezug über Lernmittelreferat des ASTA (Raum A026); nur für Mitglieder von staatlichen Hochschulen

6. C. J. Date: "An Introduction to Database Systems", Addison-Wesley, Reading, ISBN 0-201-82458-2
   Standardlehrbuch. Der Autor ist ein Verfechter des relationalen Modells. Andere Modelle werden zwar dargestellt, aber als weniger gut bewertet
7. Helmut Eirund, Ullrich Kohl: "Datenbanken leicht gemacht. Ein Arbeitsbuch für Nicht-Informatiker", Teubner-Verlag, Stuttgart, 2000, ISBN 3-519-02644-9
   Trotz des Untertitels auch für Informatiker interessant, die wenig Wert auf Theorie legen. Speziell die Strukturierung relationaler Datenbanken wird behandelt. Preis: 14,95 €
8. W. Kirsten, M. Ihringer, ...: "Objektorientierte Anwendungsentwicklung mit der postrelationalen Datenbank Caché", Springer-Verlag, Berlin, 2003, ISBN 3-540-43248-5
   Komplette Anleitung (auch für den relationalen Teil) für das DBMS Caché.
9. Michael Kofler: "MySQL: Einführung, Programmierung, Referenz", Addison-Wesley, München, 2001, ISBN 3-8273-1762-2
   Der Titel ist selbsterklärend.
10. Jay Greenspan, Brad Bulger: "MySQL/PHP-Datenbankanwendungen", mitp-Verlag, Bonn, 2001, ISBN 3-8266-0805-4
    Ein Buch, bei dem man gleich viel ausprobieren kann.
11. Kyle Geiger: "Inside ODBC", Microsoft Press, Unterschleißheim, 1996, ISBN 3-86063-359-7
    Das Buch ist verständlich und interessant. Beim Lesen ist man in der Lage, ODBC gleich zu programmieren und praktisch auszuprobieren. Die neueren Erweiterungen von ODBC fehlen natürlich.
12. Andreas Heuer: „Objektorientierte Datenbanken“, Addison-Wesley Verlag, 1997, ISBN 3-89319-800-8
    Umfassender Überblick über verschiedene Konzepte, Realisierungen und Probleme objektorientierter Datenbanken.
13. V. S. Subrahmanian: "Principles of Multimedia Database Systems", Morgan Kaufmann Publishers, San Francisco, 1998, ISBN 1-55860-466-9
    Ein detaillierter Überblick über das Gebiet.

Gliederung

1 Einführung
1.1 Erläuterung von Grundbegriffen, Aufbau eines Datenbanksystems
1.2 Mathematische Grundlagen: Relationen
1.3 Vor- und Nachteile von DBMSn

2 Strukturentwurf von Datenbanken
2.1 Überblick
2.2 Entwurfsschritte
2.3 Das Entity-Relationship-Modell
2.4 UML Klassendiagramme

3 Das relationale Datenbankmodell
3.1 Darstellung von Entities und Relationen in Tabellen
3.2 Die Relationen-Algebra
3.3 Die DB-Sprache SQL
3.4 Normalformen einer relationalen Datenbank

4 Datenbankprogrammierung
4.1 Embedded-SQL
4.2 ODBC (nur Überblick)
4.3 JDBC
4.4 Web-Anwendungen mit Datenbankzugriff, PHP (nur Überblick)

5 Das objektorientierte Datenbankmodell
5.1 Prinzipien der objektorientierten Datenbanken
5.2 Objektrelationale Datenbanken

6 Interne Funktionsweise von Datenbank-Management-Systemen
6.1 Sicherheit bei Datenbanken
6.2 Transaktionsprotokolle
6.3 Nebenläufige Verarbeitung
6.4 Datenbankintegrität: Constraints und Trigger
6.5 Abfrageoptimierung
6.6 Verteilte Datenbanken
6.7 Temporale Datenbanken
6.8 Physikalische Daten- und Speicherorganisation (als E-Learning-Modul zum Selbst-Lernen)

Regeln und Informationen zur Vorlesung

Datenbanksysteme Praktikum

Zur Vorlesung Datenbanksysteme gibt es ein Praktikum, in dem vorwiegend Übungsaufgaben mit einem Datenbankmanagement-System am Rechner gelöst werden. Das Praktikum ist dieses Semester in die Vorlesung integriert, d. h. neben ausgewiesenen Praktikumsstunden werden Übungsaufgaben zum Teil auch während der Vorlesung gestellt und bearbeitet. Für die Übungsaufgaben bekommen Sie extra Aufgabenblätter

Die Prüfungsleistung für das Praktikum ist eine praktische Arbeit; diese besteht aus den Lösungen zu bestimmten Aufgaben. Bei jedem Übungsblatt ist angegeben, für welche Aufgabe(n) die Lösung(en) abgegeben oder präsentiert werden müssen, um die praktische Arbeit zu bestehen.

Sprechzeiten

Bitte aktuelle Sprechzeiten nachsehen unter http://portal.hs-weingarten.de/web/hulin
Andere Termine bitte telefonisch vereinbaren, um E-Mail Ping-Pong zu vermeiden.

Benachrichtigungen per E-Mail

Ich werde die Teilnehmer der Vorlesung regelmäßig per E-Mail informieren, wenn sich am Vorlesungsplan etwas ändert, besondere Unterlagen mitzubringen sind, Musterlösungen neu im Internet bereit gestellt sind usw. Um diese E-Mails zu erhalten, müssen Sie im LSF die Vorlesung Datenbanksysteme belegen.

Zusatzmaterial

In der virtuellen Lernumgebung MOODLE finden Sie Material zur Vorlesung Datenbanksysteme: Skript, Übungsaufgaben, Musterlösungen, E-Learning-Module usw. Um an diese Materialien zu gelangen, müssen Sie sich bei MOODLE anmelden:

- Öffnen Sie in einem Browser die URL https://www.elearning.hs-weingarten.de/login.php oder klicken Sie im LSF bei der Veranstaltung "Datenbanksysteme", Nummer 1813 links oben auf das Moodle-Symbol (m mit Doktorhut).
- Geben Sie Ihren Hochschul-Account und Ihr Passwort an.
- In der Auflistung „Meine Kurse“ wählen Sie den Kurs „Datenbanksysteme“ durch Klick auf den Namen aus; wenn Sie über den LSF-Link kommen, sind Sie schon beim richtigen Kurs und können diesen Punkt überspringen.
- Nun haben Sie Zugriff auf alle freigegebenen Lehrmaterialien.

Hinweise und Verbesserungsvorschläge an mich

- Gerne auch **persönlich**.
- Im LSF bei der Lehrveranstaltung 1813 Datenbanksysteme rechts auf das Briefsymbol mit Stift klicken. Dadurch bekomme ich eine anonyme E-Mail mit der Anregung/Kritik.

Regeln für die Vorlesung

Damit die Vorlesung den gewünschten Lernerfolg bringt sind folgende Regeln einzuhalten:

- **Essen, Trinken** sowie die Benutzung von **Handys** ist in der Vorlesung nicht erlaubt. Während des Vorlesungsteils sollten die Rechner nicht benutzt werden, sondern nur für die Bearbeitung von Übungsaufgaben.
- **Persönliche Gesprächsrunden** innerhalb der Vorlesung sind sehr **störend** und heben den Geräuschpegel. Sie sind daher zu unterlassen. Studierende, die nach mehrmaliger Ermahnung wiederholt stören, werden von der Vorlesung **ausgeschlossen**.
- **Interaktive Teilnahme** an der Vorlesung wie z.B. Fragen zum Stoff, ergänzende Erfahrungsberichte oder Beispiele von Seiten der Studierenden sind erwünscht.

1 Einleitung

1.1 Begriffserklärung

Was ist eine Datenbank?

Beispiele: - Persönliches Telefonregister im Kalender
- Buchtitel einer Bibliothek
- Punkteregister in Flensburg

Definition:

Eine Datenbank ist eine Menge von Daten mit folgenden Eigenschaften :

- Die Daten sind dauerhaft gespeichert.
- Die Daten haben eine logische Beziehung, sie gehören zusammen.
- Sie erfüllen einen bestimmten Zweck, sie haben eine bestimmte Bedeutung für den Benutzer.
- Eine Datenbank bildet eine Abstraktion von Ausschnitten der realen Welt (Miniwelt)

Daten müssen **verwalte**t werden :

Maschinelle Verarbeitung →← Manuelle Verarbeitung
File-System →← Datenbankmanagement

Maschinelle Verarbeitung mit dem **Computer**, denn Computer rechnet nicht nur, sondern speichert auch Daten.

Aufgaben der Datenverwaltung:
Strukturdefinition, Eingabe, Ändern, Löschen und Suchen von Daten.

{Spezielle Verwaltung von Daten im Benutzerprogramm} →← {Universelle Datenverwaltungs-Programme}

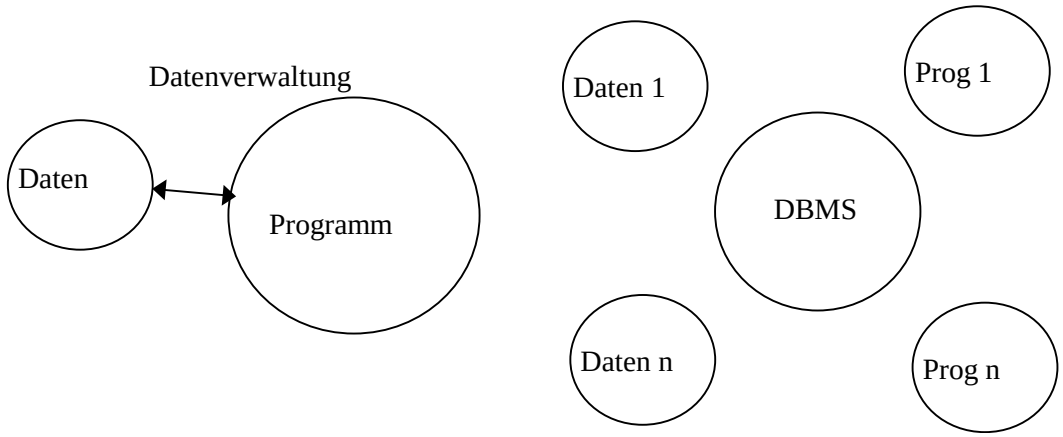


Abbildung 1: spezielle Datenverwaltung und allgemeine Datenverwaltung

Probleme bei speziellen Datenverwaltungsprogrammen:

- Mehrfache Datenspeicherung, falls mehrere Programme die gleichen Daten benötigen =>
  - Speicherverschwendung
  - Doppelte Bearbeitungszeit
  - Inkonsistente Daten
- Zugriff auf Daten des anderen Programms schwierig
- Gleichzeitiger Zugriff auf Daten → Fehler

Lösung: DBMS (Datenbank-Management-System)

Verschiedene Arten von Datenbanken

- Strukturierte Datenbanken
  - Von jedem Objekt werden ganz bestimmte Daten gespeichert
  - Viele Objekte haben die gleiche Struktur
  - Daten sind Textdaten

- Volltextdatenbanken
  - Jedes Text-Objekt besteht aus unstrukturiertem Prosatext.
  - Jeder Text kann ganz unterschiedliche Daten beinhalten.
  - Für Daten ist kein fester Platz im Dokument vorgesehen.

- Multimedia-Datenbanken
  - Jedes Objekt hat sowohl strukturierte als auch unstrukturierte Daten
  - Unstrukturierte Daten können verschiedene Medien-Daten sein: Text, Bild, Ton, . . .

Verschiedene Arten von DBMS für strukturierte Datenbanken

- Hierarchische Datenbanken
- Netzwerkdatenbanken
- Relationale Datenbanken
- Objektorientierte Datenbanken
- Objektrelationale Datenbanken
- NoSQL Datenbanken

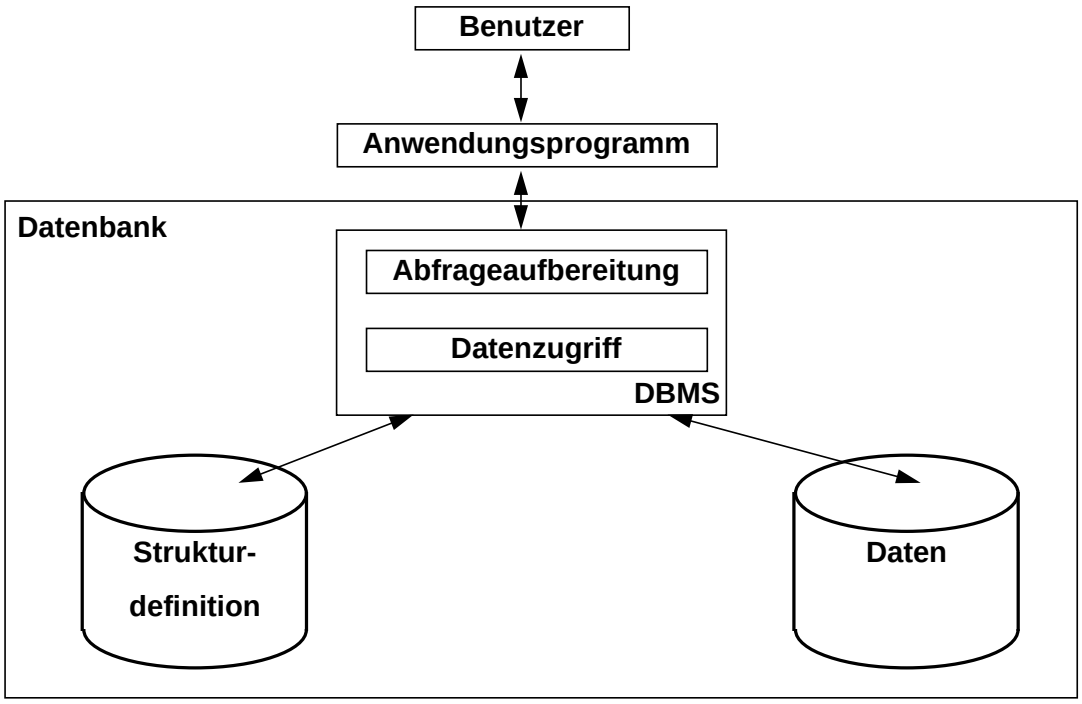


Abbildung 2: Datenfluss in einem Datenbanksystem

Datenbank Management System DBMS: Programme sowie Anwender richten Aufträge an DBMS, ohne Kenntnis über die physikalische Datenspeicherung.
Strukturdefinition (Datenbankkatalog, Data Dictionary) sagt dem DBMS, welche Daten konkret gespeichert sind und wo.

Zusätzliche Aufgaben eines DBMS

- Verwaltung von Zugriffsrechten
- Aufbereitung von Daten
- Sicherheit vor Datenverlust
- Vermeiden von Dateninkonsistenzen
- Effiziente Suche von Daten

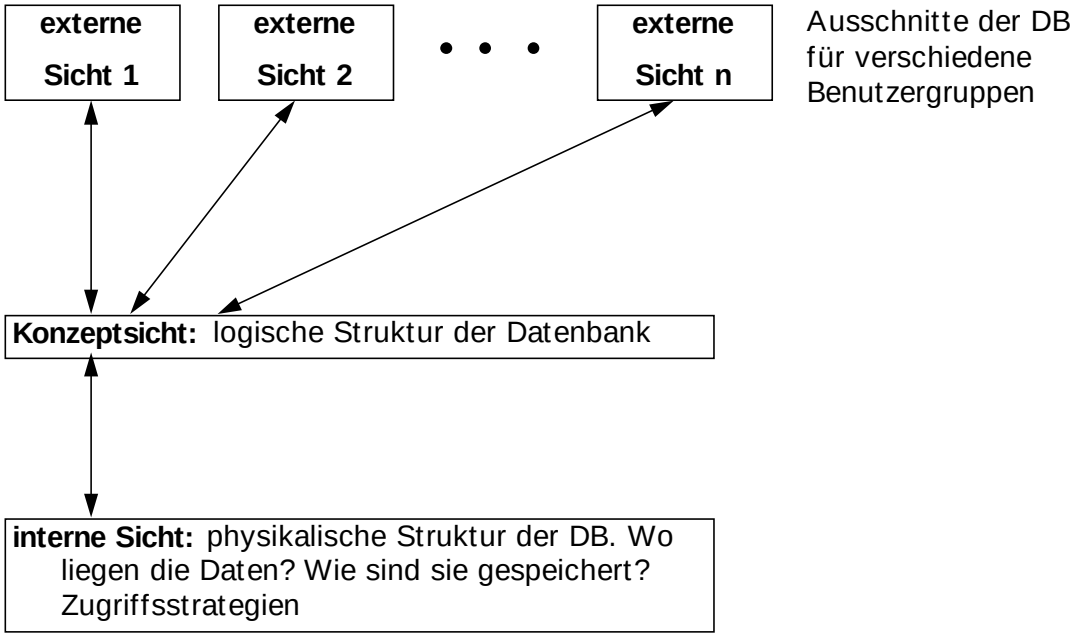


Abbildung 3: 3-Ebenen-Architektur eines Datenbanksystems

Datenunabhängigkeit :
Physikalische Datenunabhängigkeit: Physikalische Struktur der DB ist dem Benutzer verborgen !

Logische Datenunabhängigkeit: Gesamte Konzeptsicht bleibt dem Benutzer einer „externen Sicht“ verborgen.

Beispiel für Konzeptsicht und externe Sicht: Versandhaus
Kunden (Name, Adresse, Geburtsdatum, Kontonummer, BLZ, …)
Bestellungen (Auftragsnummer, Bestelldatum, Lieferdatum) mit Relation zu bestellten Artikeln
Artikel (Artikelnummer, Name, Beschreibung, Preis, Foto) mit Relation zu Lager
Lager (Ort, Kapazität, Eigenschaften)
Mitarbeiter (Name, Adresse, Geburtsdatum, Gehaltsgruppe, Steuerklasse, Aufgabengebiet, Kontonummer, BLZ, …)

Externe Sichten: Callcenter Kundenservice und Gehaltsabrechnung.

Benutzergruppen von DBMS und ihre Aufgaben :

DB-Entwickler
- Analyse der Aufgabe
- Entwurf eines Modells -> Struktur der DB
- Umsetzen in das vorhandene DBMS
- Berücksichtigung späterer Änderungen oder Zusätze

DB-Administrator (häufig identisch mit DB-Entwickler)
- Verwaltet DB (Benutzerkennungen, Anrechnung, Tuning)
- Führt Strukturänderungen durch
- Installiert neue Versionen, Tools

DB-Programmierer
- Programmiert Schablonen-Abfragen, Menüs, Tools, Bildschirmmasken,... für nicht geschulte Benutzer

DB-Benutzer
- Trägt Daten ein, löscht oder ändert Daten
- Stellt Abfragen an die DB
- Drei Gruppen :
  - Naiver oder parametrisierter Zugriff: immer gleiche oder ähnliche Zugriffe, kein Wissen über die Funktionsweise der DB
  - Seltener Zugriff, unterschiedlichste Abfragen (Manager)
  - Komplexe Zugriffe, genaue Kenntnis der DB

Datenbank speichert Daten
Benutzer muss Datenstruktur festlegen, Daten eingeben, ändern und löschen.
Benutzer sucht Daten, dazu muss der Benutzer mit der Datenbank kommunizieren
Datenbank-Sprachen dienen der Kommunikation des Benutzers mit der Datenbank

Unterscheidung bezüglich Anwendungszweck:
- Datendefinitionssprache (DDL): Strukturdefinition
- Datenmanipulationssprache (DML): Ändern, Abfrage (update, query)

Unterscheidung bezüglich Sprachebene:
- Low-Level-Sprachen verarbeiten Datensatz für Datensatz in einer vom Benutzer programmierten Schleife
- High-Level-Sprachen verarbeiten mit einem Befehl viele Datensätze
- Graphische Sprachen werden mittels einer graphischen Benutzungsoberfläche programmiert ohne dass der Anwender eine spezielle Syntax lernen muss.

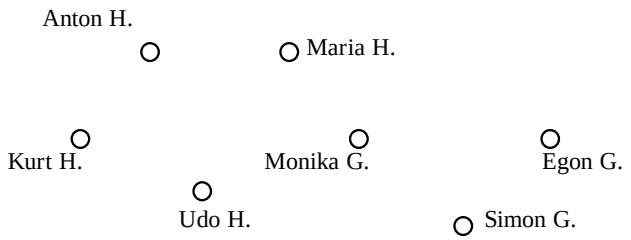
Unterscheidung bezüglich Sprachumfang:

- Reine Datenbanksprache
- Reine Datenbanksprache mit Schnittstellen zu anderen Programmiersprachen
- Programmiersprache erweitert um spezielle Konstrukte für Datenbanken
- Datenbanksprache erweitert um „normale“ Programmkonstrukte wie Schleifen

1.2 Mathematischen Grundlagen: Relationen und Graphen

Einzelne beziehungslose Daten sind sinnlos.
Die Informatik beschäftigt sich mit Beziehungen zwischen Objekten : „Relationen“

Beispiel Familie: M = Menge aller Menschen



Vater, Mutter u. Kinder bilden eine **Familie**. Ein Mensch kann zu mehreren Familien gehören. Das ist eine sehr allgemeine Art von Relation : Die Anzahl der Elemente von M, die in Beziehung stehen, ist nicht festgelegt.
Die Relation **verheiratet** ist zweistellig (= binär).

Definition Produktmenge :

Die Menge aller Paare (x, y) mit $x \in A$ und $y \in B$ heißt Produktmenge oder Kartesisches Produkt der Mengen A und B

$A \times B = \{(x,y) \ : \ x \in A, \ y \in B\}$

Definition Relation

Unter einer zweistelligen (binären) *Relation* „R“ zwischen den Mengen A und B versteht man eine Teilmenge des Kartesischen Produkts A×B: $R \subset A \times B$
Für $(x, y) \in R$ schreibt man auch $x \, R \, y$
Die Menge aller $x \in A$, die in R vorkommen, heißen *Vorbereich*
Die Menge aller $y \in B$, die in R vorkommen, heißen *Nachbereich*

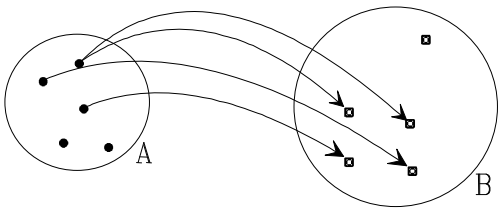
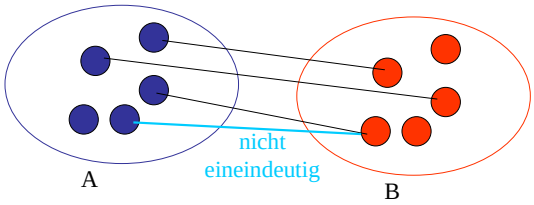


Abbildung 4: Vor- und Nachbereich einer Relation im Pfeildiagramm

Arten von Relationen

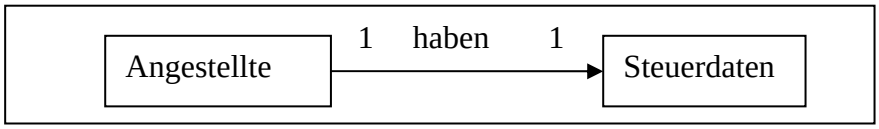
1:1 -Relation (Ein-eindeutige Relation)

In der Darstellung : Von jedem Element von A geht maximal ein Pfeil aus.
Bei jedem Element von B endet maximal ein Pfeil.



Beispiel : Angestellte (Name, Adresse, Gehalt)
Steuerdaten (Ident-Nr, Steuerklasse, ...)

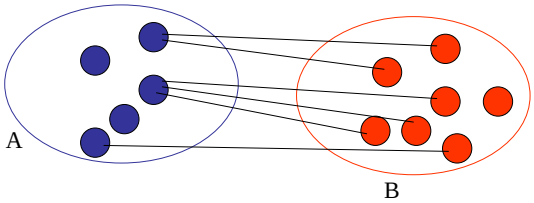
Jeder Angestellte hat nur eine Steuernummer.
⇒ Eins zu eins Relationen zwischen jedem Angestellten und seinen Steuerdaten.



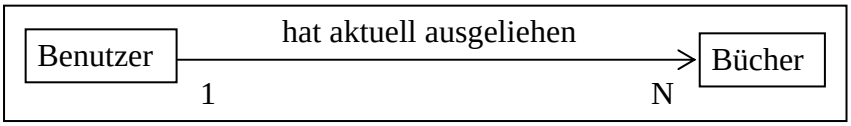
Datenstrukturdiagramm : Zwischen welchen Entities (Objektmengen) bestehen Relationen, wie heißen diese, von welcher Art sind sie ?

1:N Relationen (Voreindeutige Relationen, One-to-many Relation)

In der Darstellung : Von jedem Element von A können mehrere Pfeile ausgehen.
Bei jedem Element von B endet maximal ein Pfeil.



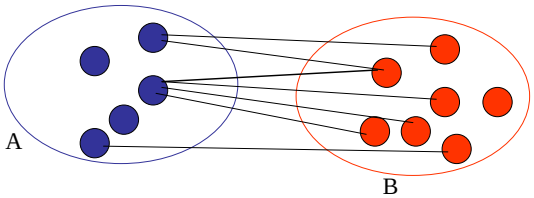
Beispiel Bibliothek: Benutzer (BenNr, Name, ...), Buch (BuchNr, Standort, . . .)



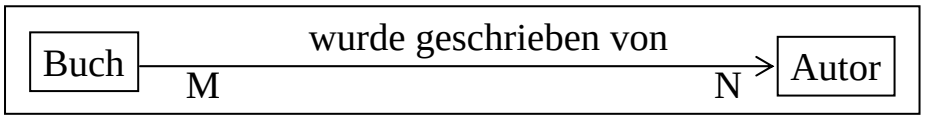
Ein Benutzer kann viele Bücher ausleihen, aber ein Buch kann zur gleichen Zeit nur von einem Benutzer ausgeliehen sein.

M:N Relationen (Mehrdeutige Relationen)

In der Darstellung: Von jedem Element von A können mehrere Pfeile ausgehen.
Bei jedem Element von B können mehrere Pfeile enden.



Beispiel: „Buch wurde geschrieben von Autor“



Ein Autor kann mehrere Bücher schreiben, ein Buch kann mehr als einen Autor haben.

Übung in der Vorlesung: Welche der präsentierten Relationen sind eineindeutig, voreindeutig (in welcher Richtung) oder mehrdeutig?

Speicherung von Relationen mit unbegrenzter Stelligkeit
Objektifizierung der Relation und Umwandlung in 2- stellige Relation:
Jede Gruppe von Elementen, die in Relation stehen, wird ein eigenes Objekt. So entsteht eine Menge von Relationsobjekten. Im Beispiel Familie entsteht die Menge aller Familien, d. h. jede Familie wird eigenes Objekt. Zwischen der Menge M der Menschen und der Menge der Familien gibt es dann eine 2-stellige Relation „gehört zu“.

Mensch	Familie
(Anton H.,	Huber)
(Maria H.,	Huber)
(Monika G.,	Huber)
(Monika G.,	Graf)
(Egon G.,	Graf)

1.3 Zusammenfassung

Datenbank: Zusammengehörende Daten repräsentieren Ausschnitt und Abstraktion der realen Welt (Miniwelt). Sie werden permanent gespeichert.

Verwaltung der Daten von jeder Anwendung separat (auf File) oder für alle Programme von einem **Datenbankmanagementsystem**.

Vorteile von DBMS
- Kontrollierte Redundanz, Konsistenz von Daten
- Mehrfachnutzung von Daten möglich
- Schutz vor unberechtigtem Zugriff
- Einfache Darstellung komplexer Beziehungen zwischen den Daten
- Integritätsbedingungen helfen, falsche Eingaben zu entdecken.
- Sicherheit gegen Datenverlust durch automatische Sicherungskopien
- Umfangreiche Suchfunktionen

Nachteile von DBMS
- Zusätzliche Kosten
- Mehr Bedarf an Ressourcen (Speicher, Prozessorleistung)
- Rechenzeitverlust für zusätzliches Programm
- Einarbeitungszeit für den Benutzer

Daher Verzicht auf DBMS bei
- einfachen Anwendungen mit nur einem Benutzer (zur gleichen Zeit), weil der Mehraufwand für DBMS in keinem Verhältnis zum Nutzen steht.
- Realzeitanforderungen, die unbedingt eingehalten werden müssen, weil der Rechenzeitverlust durch den Aufruf eines zusätzlichen Programmsystems nicht in Kauf genommen werden kann.

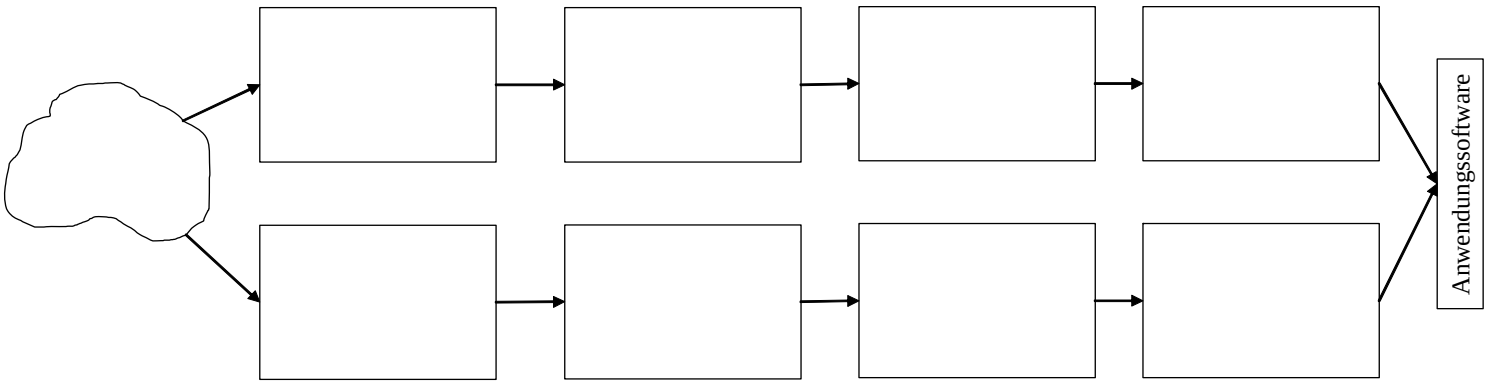
2 Modelle zum Datenbankentwurf

2.1 Überblick ER-Modell (Entity-Relationship-Modell)

- Einsatz für den logischen Datenbankentwurf bei strukturierten Datenbanken
- Ziel ist ein gut strukturierter, normalisierter DB-Entwurf
- Grafische Veranschaulichung, CASE-Tool
- Zeigt semantische Information über den Inhalt und den Zusammenhang von Daten.
- Besteht aus Entities und Relationen zwischen Entities.
- Entity = Objekt (physikalisch, gedacht) der realen Welt
- Entity-Set = Objektmenge gleichartiger Objekte
- Erfinder: Chen, Peter (Chen Pin-Shan 陳品山): (1976 *The Entity Relationship Model – Toward A Unified View of Data*)
- Erweiterung im Enhanced-Entity-Relationship-Modell (EER-Modell)

2.2 Entwurfsmethode

- Analyse der Aufgabenstellung
- Führe parallel zur Datenmodellierung auch eine Prozessmodellierung durch, die die Datenmodellierung beeinflusst.
- Von welchen Objekten werden Daten (Attribute) benötigt?
- Fasse Objekte mit gleichen Attributen zu Objektmengen zusammen: Entity-Sets
- Alle Elemente eines Entity-Sets haben die gleiche Attribut-Struktur: Entity-Typ
- Welche Relationen gibt es zwischen den Entities?
- Von welcher Art sind die Relationen?



Entwurfsschritte einer Datenbankanwendung ----> beeinflusst

2.3 Semantische Elemente des ER-Diagramms

Entity-Sets (Entity-Mengen) und Entity-Typen mit Attributen

Relationen

Beispiel: ER-Diagramm für eine Fachhochschule (wird in der Vorlesung entwickelt)
Genauere Kennzeichnung der Art von Relationen http://de.wikipedia.org/wiki/Min-Max-Notation

Relationen mit eigenen Attributen

Auch Relationen können eigene Attribute erhalten. In vielen Fällen könnte ein Attribut nirgends sonst untergebracht werden.

Relationen umwandeln in Entity-Sets

Jede M:N-Relation kann in ein Entity-Set und zwei 1:N-Relationen umgewandelt werden.

Eine **weak entity** ist eine Entity, die für sich allein keinen Sinn macht. Man braucht die Zuordnung zu anderen Entities, um die Daten sinnvoll interpretieren zu können. Z. B. macht eine Note keinen Sinn, wenn man nicht weiß, wer die Note in welchem Fach erzielt hat.

Drei- und mehrstellige Relationen

Beispiel einer mehrstelligen Relation: Die mehrstellige Relation „Stundenplan“ kann nicht durch mehrere 2-stellige Relationen zwischen den beteiligten Entities ersetzt werden. Sie wird durch Objektifizierung zu einer eigenen Tabelle „Stundenplan“ und jeweils 1:N-Relationen zu den beteiligten Entities implementiert.

Mehrwertige Attribute

Die Datenwerte eines Attributes einer Entity können eine Menge mehrerer Werte sein.
Beispiel: Ausstattung (Geräte) in Hörsälen

Zusammengesetzte Attribute

Ein Attribut besteht aus mehreren Teilen. Beispiel: Name und Adresse

Spezialisierung in Subklassen, Generalisierung zu Superklassen

Wie in der objektorientierten Programmierung üblich, kann ein Entity-Set = Klasse aus mehreren Unterklassen bestehen. Beispiel: Personen, Studenten und Dozenten.

Umgekehrt können mehrere Entity-Sets unter einem neuen Gesichtspunkt zu einer Oberklasse zusammengefasst werden.

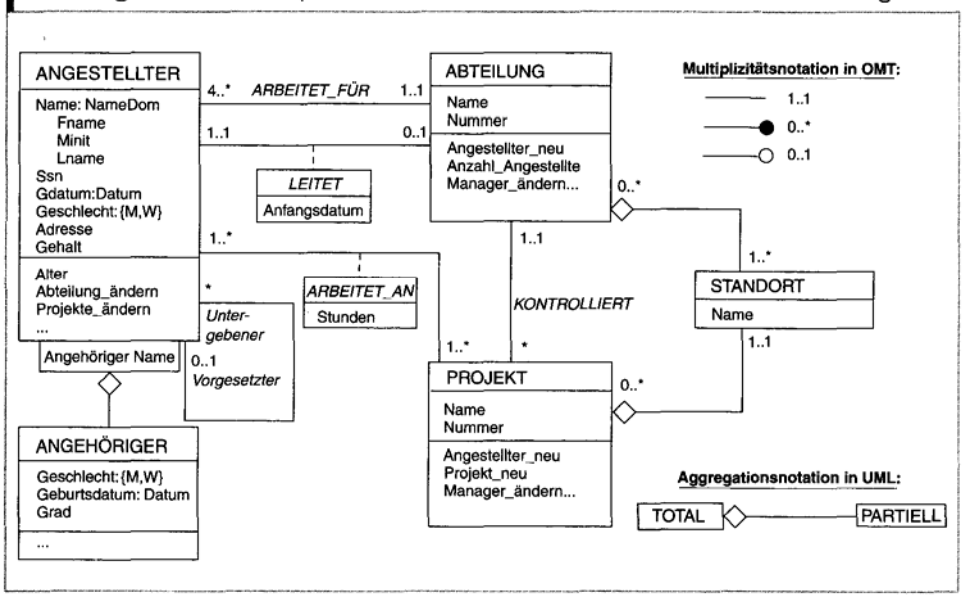
Beispiel: Firma verkauft Autos und Reisen, beides sind Kaufobjekte

2.4 Konzeptuelle Objektmodellierung mittels UML Klassendiagrammen

Kopie aus R. Elmasri, S. Navathe: "Grundlagen von Datenbanksystemen", Pearson Studium München, 2002, Kapitel 4.6 (S. 116 – 119). Alle Verweise, Kapitelnummern, Abbildungsnummern usw. in diesem Abschnitt beziehen sich auf dieses Buch.

Objektmodellierungsmethoden wie UML (Universal Modeling Language) und OMT (Object Modeling Technique) werden immer häufiger eingesetzt. Obwohl diese Methoden hauptsächlich für den Softwareentwurf entwickelt wurden, umfasst ein großer Teil des Softwareentwurfs den Entwurf von Datenbanken, auf die verschiedene Programme zugreifen. Folglich ähnelt ein wichtiger Teil dieser Methoden, d.h. die Klassendiagramme, auf vielerlei Art den EER-Diagrammen. Leider unterscheidet sich oft die Terminologie. In diesem Abschnitt beschreiben wir kurz einen Teil der Notation sowie Fachbegriffe und Konzepte der UML-Klassendiagramme und vergleichen sie mit der EER-Terminologie und -Notation. Abbildung 4.11 zeigt, wie das ER-Datenbankschema FIRMA aus Abbildung 3.15 in der UML-Notation dargestellt werden kann. Die Entitätstypen aus Abbildung 3.15 sind in Abbildung 4.11 als Klassen modelliert. Eine Entität im ER-Schema entspricht einem Objekt in UML.

Abbildung 4.11: Das konzeptuelle UML-Schema für die Datenbank FIRMA aus Abbildung 3.15.



In UML-Klassendiagrammen wird eine Klasse als Rechteck (siehe Abbildung 4.11) dargestellt, das (von oben nach unten) die drei folgenden Abschnitte umfasst: Klassenname, Attribute für einzelne Objekte der Klasse und Operationen, die auf diese Objekte ausgeführt werden können. Operationen werden in EER-Diagrammen nicht spezifiziert. Man betrachte die Klasse ANGESTELLTER in Abbildung 4.11. Ihre Attribute sind Name, Sozialversicherungsnummer (Ssn), Geburtsdatum (Gdatum), Geschlecht, Adresse und Gehalt. Dem Attributnamen folgt — durch Doppelpunkt getrennt — der Wertebereich (Domäne) für das Attribut bzw. die Aufzählung möglicher Werte (siehe die Attribute Name, Geschlecht und Geburtsdatum von ANGESTELLTER in Abbildung 4.11). Ein zusammengesetztes Attribut wird als strukturierter Bereich modelliert, wie das Attribut Name von ANGESTELLTER verdeutlicht. Ein mehrwertiges Attribut wird im Allgemeinen als separate Klasse modelliert, wie z.B. die Klasse STANDORT in Abbildung 4.11.

Beziehungstypen werden in der UML-Terminologie als Assoziationen und Beziehungsinstanzen als Verknüpfungen bezeichnet. Eine binäre Assoziation (binärer Beziehungstyp) wird als Linie dargestellt, die die teilnehmenden Klassen (Entitätstypen)

verbindet, und darin (wahlweise) einen Namen haben. Ein Beziehungsattribut, das als Verknüpfungsattribut bezeichnet wird, wird in einem Rechteck eingefügt, das mit der Linie der Assoziation durch eine gestrichelte Linie verbunden ist. Die in Abschnitt 3.7.4 beschriebene (min. max)-Notation wird benutzt, um Beziehungseinschränkungen zu spezifizieren, die in der UML-Terminologie als Multiplizitäten bezeichnet werden. Multiplizitäten werden in Form von min...max spezifiziert und ein Sternchen (*) bedeutet, dass die Teilnahme keine Obergrenze hat. Allerdings werden die Multiplizitäten an den entgegengesetzten Enden der Beziehung angeordnet, im Vergleich zu der in Abschnitt 3.7.4 beschriebenen Notation (man vergleiche die Abbildungen 4.11 und 3.15). In UML bedeutet ein einzelnes Sternchen eine Multiplizität von O...* und eine einzelne 1 bedeutet eine Multiplizität von 1...1. Eine rekursive Beziehung (siehe Abschnitt 3.4.2) wird in UML als reflexive Assoziation bezeichnet und die Rollennamen — ebenso wie die Multiplizitäten — werden im Vergleich zu den Rollennamen in Abbildung 3.15 an den entgegengesetzten Enden einer Assoziation angeordnet.

In UML gibt es zwei Arten von Beziehungen: Assoziation und Aggregation. Eine Aggregation dient der Darstellung einer Beziehung zwischen einem (aggregierendem) Objekt und seinen Komponenten, die ebenfalls durch eine Diagrammnotation dargestellt wird. In Abbildung 4.11 haben wir die Standorte einer Abteilung und den einzelnen Standort eines Projekts als Aggregationen modelliert. Aggregation und Assoziation haben aber keine unterschiedlichen strukturellen Eigenschaften, die Wahl zwischen den beiden Beziehungsarten ist in gewissem Sinn subjektiv. Im EER-Modell werden beide als Beziehungen dargestellt. UML unterscheidet auch zwischen unidirektionalen Assoziationen/Aggregationen, die mit einem Pfeil dargestellt werden, um anzudeuten, dass nur eine Richtung für den Zugriff auf zusammenhängende Objekte nötig ist, und bidirektionalen Assoziationen/Aggregationen, dem Standardfall (Default) in UML. Zusätzlich können Beziehungsinstanzen als geordnet spezifiziert werden. Beziehungs- bzw. Assoziationsnamen sind in UML optional. Beziehungsattribute werden in einem Rechteck dargestellt, das mit einer gestrichelten Linie mit der Linie verbunden wird, die die Assoziation/Aggregation darstellt (siehe Anfangsdatum und Stunden in Abbildung 4.11).

Die in jeder Klasse angegebenen Operationen werden von den funktionalen Anforderungen der Anwendung abgeleitet, wie in Abschnitt 3.1 beschrieben. Im Allgemeinen reicht es aus, anfangs die Operationsnamen für die Operationen zu spezifizieren, die erwartungsgemäß auf die einzelnen Objekte einer Klasse ausgeführt werden sollen, wie in Abbildung 4.11 dargestellt. Mit zunehmender Verfeinerung des Entwurfs werden weitere Details hinzugefügt, z.B. die genauen Argumenttypen (Parameter) für jede Operation und eine funktionale Beschreibung jeder Operation. UML hat Funktionsbeschreibungen und Sequenzdiagramme, um einige Operationsdetails zu spezifizieren, diese gehen jedoch über Zweck und Umfang dieses Buchs hinaus und sind in der einschlägigen Fachliteratur nachzulesen.

Schwache Entitäten können in UML mit Hilfe des Konstrukts qualifizierte Assoziation (oder qualifizierte Aggregation) modelliert werden. Dies kann sowohl die identifizierende Beziehung als auch den partiellen Schlüssel, der in einen mit der Eigentümerklasse verbundenen Kasten eingefügt wird, darstellen. Dies wird in Abbildung 4.11 durch die Klasse ANGEHÖRIGER und ihre qualifizierte Aggregation zu ANGESTELLTER dargestellt.

Abbildung 4.12: Spezialisierung/Generalisierung in UML-Notation, dargestellt durch ein Klassendiagramm, das dem EER-Diagramm in Abbildung 4.7 entspricht.

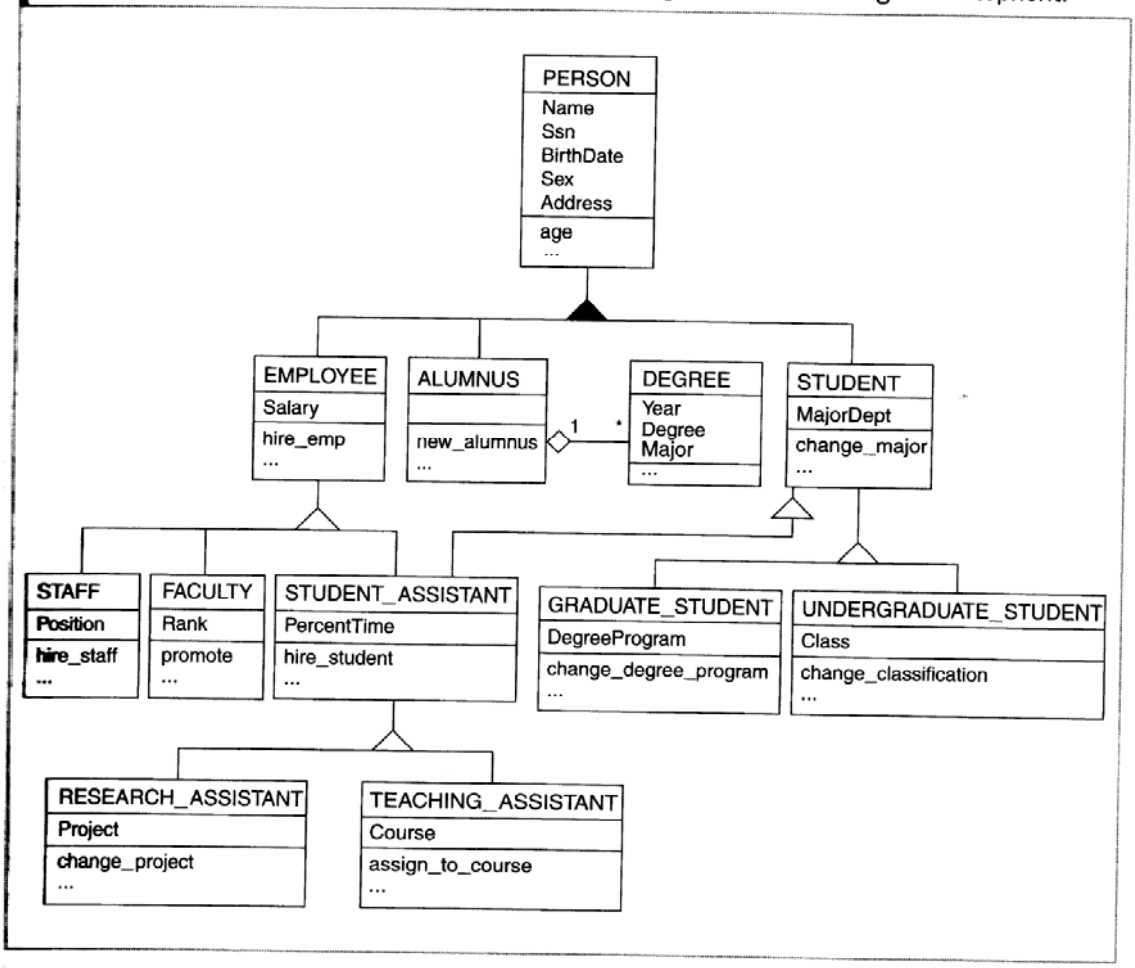
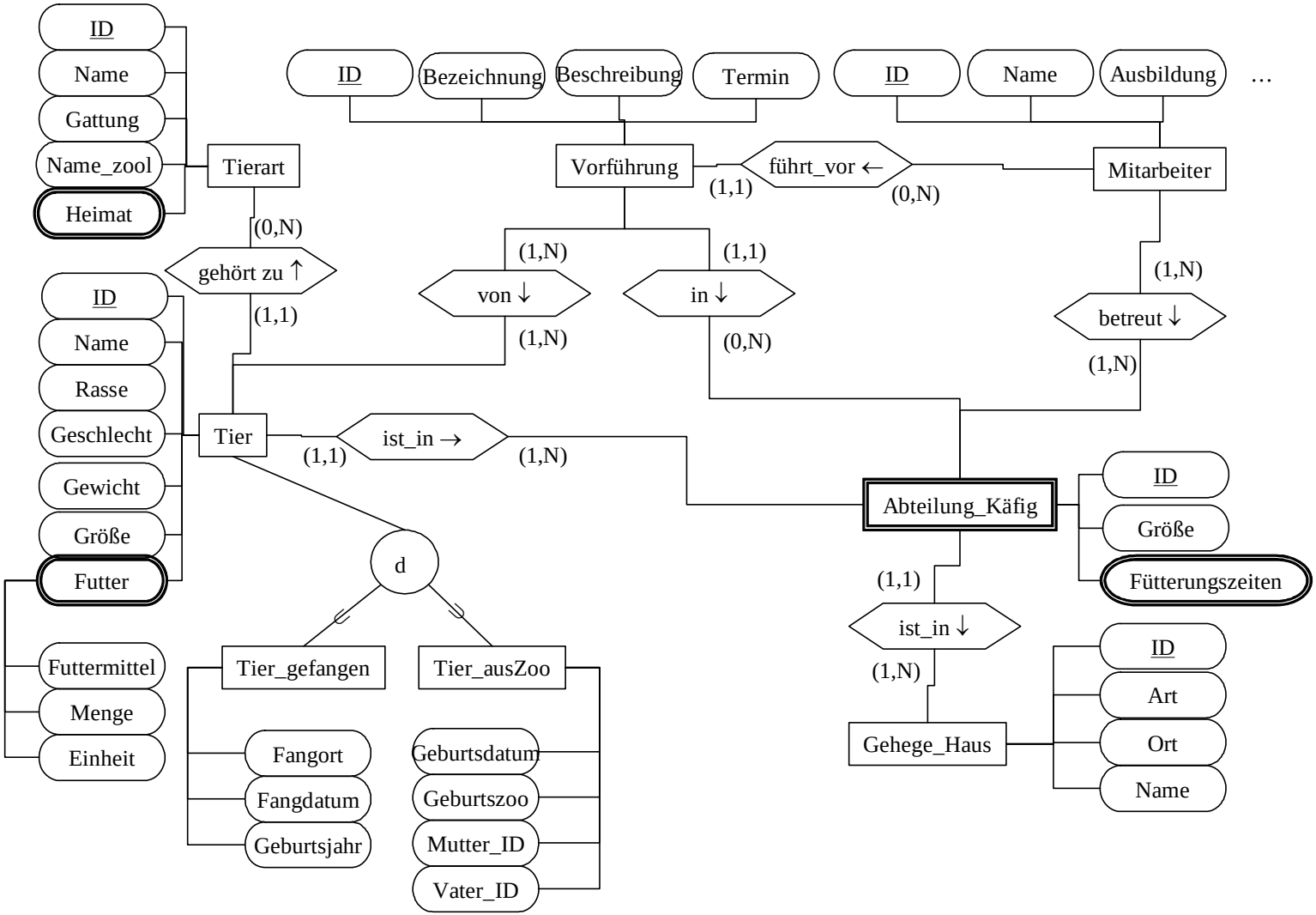


Abbildung 4.12 zeigt die UML-Notation für Generalisierung/Spezialisierung anhand eines UML-Klassendiagramms, das dem EER-Diagramm aus Abbildung 4.7 entspricht. Ein leeres Dreieck bedeutet eine disjunkte Spezialisierung/Generalisierung und ein gefülltes Dreieck eine überlappende.

Die obigen Beschreibungen und Beispiele bieten einen kurzen Überblick über UML-Klassendiagramme und -Begriffe. Viele Details wurden nicht berücksichtigt, weil sie über Zweck und Umfang dieses Buchs hinausgehen. Das Literaturverzeichnis am Ende dieses Kapitels enthält einige Referenzen auf Bücher, in denen UML umfassend beschrieben wird.

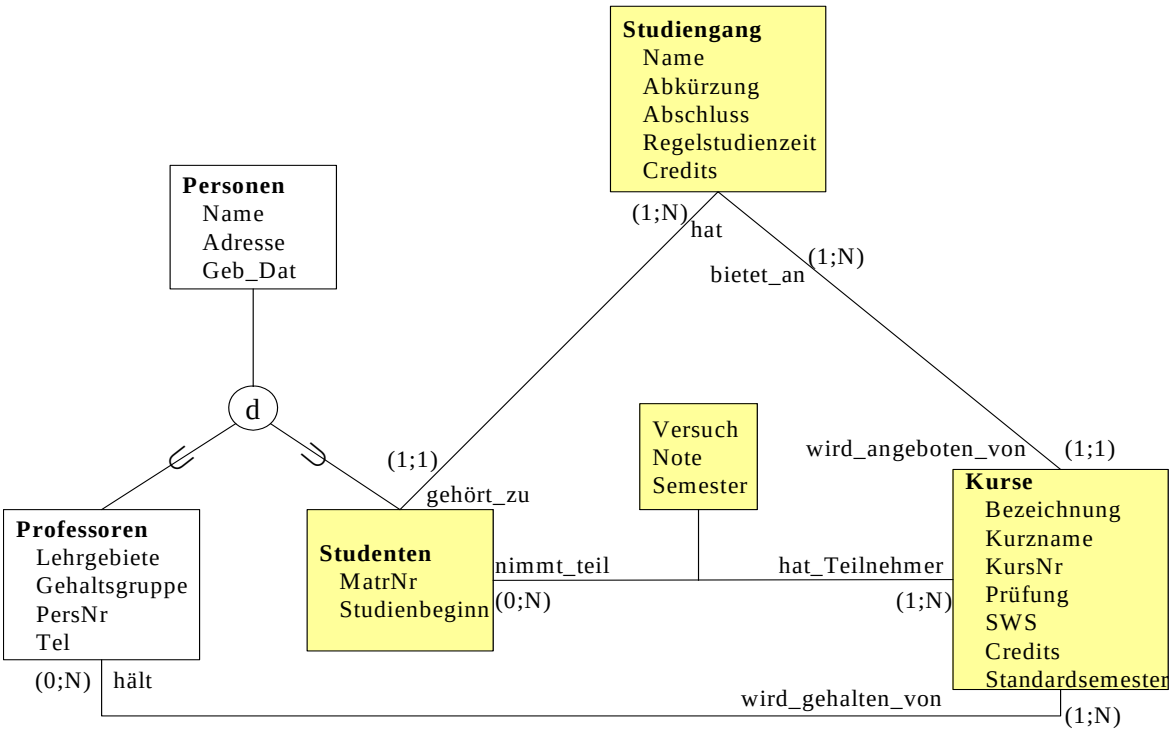


Beispiel: ER-Diagramm "Zoo" (aus Klausur WS 2001/2)

3 Das Relationale DB - Modell

3.1 Darstellung von Daten und Relationen

Erfunden von E. F. Codd 1970: „A Relational Model for Large Shared Data Banks“
Beispiel (Ausschnitt ER-Diagramm Hochschule Ravensburg-Weingarten**):**



Im Relationalen DB - Modell, werden alle Daten und Relationen in **Tabellen** gespeichert.
Ebenso Strukturdefinitionen und andere Metainformationen.
Jedes Entity-Set eines Entity-Relationship-Modells wird in eine Tabelle umgewandelt.
Eine Tabelle ist eine Matrix mit Zeilen und Spalten.
Jedes Attribut einer Entity wird zu einer Spalte der Tabelle.
Jedes konkrete Objekt eines Entity-Sets wird als Datensatzzeile in der Tabelle gespeichert.

Schema: Name der Tabelle, Attribute mit Datentypen = Rahmen der Tabelle = Leerformular

Instanz: Tabellen mit konkreten Datensätzen → ausgefülltes Formular

Studiengang

Name	Abkürzung	Abschluss	Regelstudienzeit	Credits
Angewandte Informatik Bachelor	AI B	BSc	6 Semester	180
Wirtschaftsinformatik Bachelor	WI B	BSc	7 Semester	210
Technik-Management Bachelor	TM B	BSc	7 Semester	210
Informatik Master	IN M	MSc	4 Semester	120

Studenten

Name	Adresse	Geb_Dat	MatrNr	Studienbeginn	
Abel	Südstr. 1	7.11.1985	12345	1.10.2004	
Kuhn	Ostweg 2	1.1.1986	11122	1.10.2004	
Vogel	Südstr. 5	21.5.1984	34343	1.03.2004	

Kurse

Bezeichnung	Kurzname	KursNr	PrüfArt	…
Digitaltechnik u. elektr. Grl.	DIGT	1438	K 60	
Lineare Algebra	LINAL	1407	K 60	
Datenbanksysteme	DABS	1813	K 90	
Programmieren Praktikum	PROGP	1803	PA	

Tabellen sind an sich Darstellungen von Relationen. z.B. $\textit{PrüfArt} \xleftrightarrow[1:N]{} \textit{Bezeichnung}$ in Kurse

$\subset$ Menge d. Kursbezeichnungen $\times$ Menge d. Prüfungsleistungen
Tabellen werden daher auch **Basisrelationen** genannt.

Darstellung von 1:N - Relationen zwischen verschiedenen Tabellen :

Schlüssel der 1-Teil-Tabelle, wird als zusätzliches Attribut bei der N-Teil-Tabelle wiederholt. Damit dienen Dateneinträge als implizite Zeiger. (siehe Beispiel: Attribut *Studiengang* bei Tabelle *Studenten*)

Dabei bezeichnet die Abkürzung des Studiengangs in der Tabelle Studiengang eindeutig genau einen Studiengang. Es gibt **keine** zwei Studiengänge mit der gleichen Abkürzung. Die Abkürzung ist ein **Schlüssel** der Tabelle Studiengang. Beim Student wird die Abkürzung des Studiengangs-namen wiederholt, sie zeigt auf das Schlüsselfeld einer fremden Tabelle, sie ist ein **Fremdschlüssel**.

Definition verschiedener Schlüsselarten

Ein **Superschlüssel (super key)** ist eine Kombination von Attributen, die jeden Datensatz der Tabelle eindeutig bestimmen. Es gibt keine 2 Datensätze, die in diesen Attributwerten übereinstimmen. Z.B. ist die Gesamtheit aller Attribute einer Tabelle ein Superschlüssel, falls keine doppelten Datensätze vorkommen.
Beispiel:

Ein **Minimalschlüssel (candidate key)** ist ein Superschlüssel, aus dem kein Attribut weggelassen werden darf, ohne daß die Schlüsseleigenschaft verloren ginge.
Beispiel:

Ein **Primärschlüssel (primary key)** ist ein Schlüssel, der aus allen Minimalschlüsseln vom DB-Designer ausgewählt wurde.
Beispiel:

Ein **Fremdschlüssel (foreign key)** ist ein Attribut einer Tabelle, das sich auf den Primärschlüssel einer anderen Tabelle bezieht. Damit werden die impliziten Verweise bei 1:N-Relationen implementiert.
Beispiel:

Bei 1:1 - Relationen existieren 2 Vorgehensweisen :

- Gleiches Vorgehen wie bei 1:N Relationen mit N = 1
- Zusammenfassen von 2 Tabellen zu einer (sinnvoll, wenn die Relation eine Funktion ist)

Darstellung von M:N - Relationen (schwierig) :

Beispiel : $Studenten \xleftrightarrow[M:N]{} Kurse$

1. Möglichkeit: Supertabelle, d. h. eine Tabelle mit allen Studenten- und Kursdaten.
Folge : Daten eines Studenten sind redundant vorhanden, nämlich für alle seine Kurse. Ebenso sind alle Kursdaten für jeden Kursteilnehmer aufgeführt. Durch die Redundanz entstehen Anomalien (siehe Kapitel 3.4.1 Informelle Regeln für RDBS)

Supertabelle Studenten-Kurse

SName	Adresse	Geb_Dat	MatrNr	...	KursNr	KursName	PrüfArt	...
Abel	Südstr. 1	18.10.1985	12345		1803	PROGP	PA	
					1438	DIGT	K 60	
Kuhn	Ostweg 2	1.1.1986	11122		1803			
Vogel	Südstr. 5	21.5.1984	343434		1407			

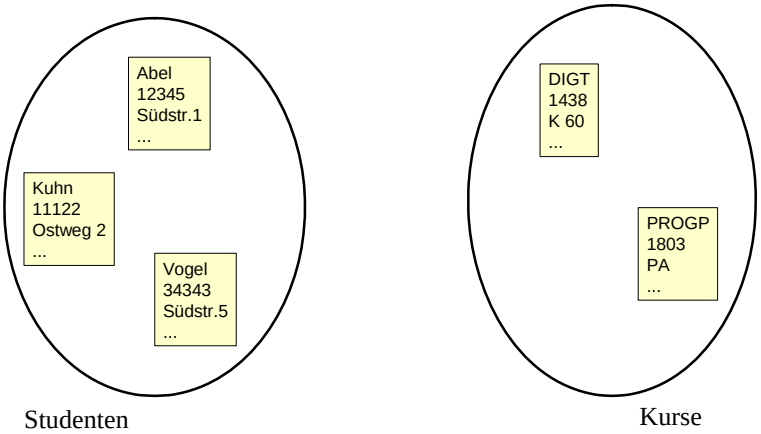
2. Möglichkeit :
Wie bei der 1:N Relation wird der Schlüssel der einen Tabelle in der anderen wiederholt.

SName	Adresse	Geb_Dat	MatrNr	...	KursNr
Abel	Südstr. 1	18.10.1985	12345		1803
					1438
Kuhn	Ostweg 2	1.1.1986	11122		1803
Vogel	Südstr. 5	21.5.1984	34343		1438

KursName	Kurs Nr	Prüfungs-leistung
DIGT	1438	K 60
PROGP	1803	PA

Folge : Redundanz

3. Möglichkeit : Verbindungstabelle (connection table)
z.B. MatrNr „nimmt teil“ KursNr



MatrNr	KursNr	Note	Versuch	Semester
12345	1438	2,0		
12345	1803	BE		
11122	1803	BE		
34343	1438	2,3		

Die Verbindung besteht aus den Schlüsseln der Tabellen, die in der M:N-Relation stehen (Fremdschlüssel), sowie eventuell anderen Attributen, die der Relation zugeordnet sind.

3.2 Übersetzung eines Entity-Relationship-Modells in ein relationales Datenbanksystem :

a) Jedes Entity-Set wird eine Tabelle mit den Attributen als Spalten und einem Primärschlüssel
b) Bei 1:N und 1:1 Relationen, wird der Primärschlüssel des 1-Teil beim N-Teil als Fremdschlüssel wiederholt.
c) Jede M:N-Relation wird zu einer eigenen Verbindungstabelle; Attribute sind die Primärschlüssel der Entities, die in Relation stehen, als Fremdschlüssel.
d) Jedes mehrwertige Attribut wird eine eigene Tabelle.
e) Bei Entities mit Unterentities gibt es drei Implementierungs-Alternativen:
   - Es gibt nur die Tabelle für das Oberentity-Set. Eine zusätzliche Spalte kennzeichnet, zu welcher der Unterentity-Sets ein konkretes Objekt gehört.
   - Für jedes Unterentity-Set gibt es eine eigene Tabelle. Der Zusammenhang zwischen den Tabellen ist nicht sichtbar außer über gleichnamige Spalten.
   - Es gibt eine Tabelle für das Oberentity-Set mit allen Attributen des Oberentity-Sets als Spalten und zusätzlich je eine Tabelle für jedes Unterentity-Set mit den jeweiligen speziellen Spalten als Attribute. Mittels eines Fremdschlüssel verweist jeder Datensatz aus den untergeordneten Tabellen auf den zugehörigen Teil in der Tabelle für das Oberentity-Set.

Beispiel: Personen, Studenten und Professoren

3.3 Die Sprache SQL

SQL = Structured Query Language, die Standard-DB-Sprache

SQL ist beeinflusst von der Relationenalgebra, hat aber eine andere Syntax. Sie ist textgebunden (nicht grafisch), tabellenorientiert (nicht datensatzorientiert) und soll durch Anlehnung an die natürliche englische Sprache leicht zu verstehen sein. Sie kann für sich als interaktive DB-Sprache oder eingebettet in eine gängige Programmiersprache (Pascal, C, JAVA ...) benutzt werden.

3.3.1 Datendefinition

- **Tabelle erstellen (die Struktur): Datendefinition**
  ```
  CREATE TABLE <Name> (<Spaltenbezeichnung> <Feldtyp>, ...);
  ```
  Bespiel: Erstelle Tabelle Studiengang

  _____

  _____

  _____

  _____

  _____

  _____

  _____

  _____

  * Teile einer **Tabelle** in eine neue Tabelle **kopieren** mit SELECT (siehe 3.3.4)
  ```
  CREATE TABLE WIStud  AS
  SELECT Name, Adresse FROM Studenten WHERE Studiengang =
  'WIB';
  ```

  * Fremdschlüssel definieren: z. B. Tabelle Studenten mit Bezug zu einem Studiengang
  ```
  CREATE TABLE Studenten
  (Name          VARCHAR (50),
   Adresse       VARCHAR (100),
   Geb_Dat       DATE,
   Geschlecht    CHAR (1),
   MatrNr        INTEGER _____

   Studienbeginn DATE,
   Studiengang   CHAR(3) _____

  );
  ```

- **Tabelle löschen**
  ```
  DROP TABLE <Name>;
  ```

  _____

- **Tabelle ändern**

```
ALTER TABLE <Name> ADD\MODIFY (<Spalte> <Typ>, ...);
```

Beispiel 1: Füge ein Attribut zur Professoren-Tabelle hinzu

Beispiel 2: Deklariere nachträglich einen Primärschlüssel

Beispiel 3: Deklariere nachträglich einen Fremdschlüssel mit oder ohne Löschweitergabe

Weitere Beispiele zum Einfügen von Constraints :

Beispiel 4 : (direkt beim create table)

```
CREATE TABLE master (
 a1 char (5) CONSTRAINT pk_master PRIMARY KEY,
 a2 char (10),
 a3 char (10) NOT NULL);

CREATE TABLE  slave1 (
 a1 char (5) CONSTRAINT fk_a1 REFERENCES master(a1),
 a2 char (10));
```

Beispiel 5 : Im nachhinein, das Verbindungsattribut wird später zugefügt

```
CREATE TABLE slave2 (
 a1 char (10),
 a2 char (10));

ALTER TABLE slave2 ADD
(a3 char (5) CONSTRAINT  fk_slave2 REFERENCES  master(a1));
```

Beispiel 6: Im nachhinein, nur Constraint

```
CREATE TABLE slave3 (a1 char (10), a2 char (10), a3 char (5));

ALTER TABLE slave3 ADD
(CONSTRAINT fk_slave3 FOREIGN KEY (a3) REFERENCES master
(a1));
```

Verwendung des Kommandos "Alter Table"

- fügt Attribute hinzu oder löscht Attribute
  Beispiel:_____

  _____

- vergrößert Spalten
  Beispiel:_____

  _____

- fügt Constraints hinzu oder löscht Constraints
  Beispiel:_____

  _____

- schaltet Constraints ein und aus
  Beispiel:_____

  _____

- ändert Tabellennamen
  Beispiel:_____

  _____

3.3.2 Datenmanipulation schreibend

- **Einfügen Datensatz**

```
INSERT INTO <Tabelle> VALUES
(<Wert Spalte 1>, <Wert Spalte 2>, ...);
```

Beispiel: Füge neuen Professor ein

- **Kopieren von Datensätzen aus anderen Tabellen mit SELECT (siehe 3.3.4):**

```
CREATE TABLE Adressliste (Name CHAR (50), Adresse CHAR (100));
INSERT INTO Adressliste SELECT Name, Adresse FROM Professoren;
INSERT INTO Adressliste SELECT Name, Adresse FROM Studenten;
```

- **Löschen Datensatz**

```
DELETE FROM <Tabelle> WHERE <Bedingung>;
```

Beispiel: Lösche einen Kurs

- **Ändern Datensatz**

```
UPDATE <Tabelle> SET <Spalte> = <New> WHERE <Bedingung>;
```

Beispiel: Ändere die Semesterwochenstunden von GRABO

3.3.3 Datenmanipulation lesend: Grundlage Relationenalgebra

Der lesende Zugriff auf Datenbanken (Abfragen) mit SQL baut auf der Relationenalgebra auf, hat aber eine andere Syntax. Die Relationenalgebra dient zum Bearbeiten und Verknüpfen von Relationen, ist aber auch eine DB-Sprache.

Jede Tabelle stellt Relationen dar; Spalten werden als Grundmengen aufgefasst.

$T \subset M_1 \times M_2 \times M_3 \times ... \times M_n$

Beispiel : Arbeitet an $\subset$ PersNr $\times$ ProjNr $\times \{std \in \mathbb{N}: 1 \leq std \leq 40\}$

Relationenalgebra:
Mengen zusammen mit Verknüpfungen und Verknüpfungsregeln, heißen algebraische Strukturen = Algebren. Hier: Relationenalgebra, da Relationen verknüpft werden.

Definition verträglich:
Zwei Relationen (Tabellen) T_1 und T_2 heißen **verträglich**, wenn sie gleich viel Spalten haben und die Datentypen jeweils übereinstimmen.

Verknüpfungen von Relationen :
Beispieltabellen:

nimmt_teil

MatrNr	KursNr	Note	Versuch	Semester
11122	1803	BE	1	WS 04/05
11122	9999	2,0	1	SS 03
12345	1438	2,0	1	WS 03/04
12345	1803	BE	1	WS 04/05
22111	1803	BE	1	SS 04
22111	1438	2,3	1	WS 04/05
22111	9999	3,0	1	SS 03
343434	1803	BE	1	WS 03/04
343434	1111	3,0	1	SS 04
987654	1803	BE	1	WS 04/05
987654	1111	1,0	1	WS 04/05
987654	1438	1,3	1	SS 04

Studentinnen

Name	Adresse	Geb_Dat	MatrNr	Studiengang
Kuhn	Ostweg 2	18.10.1985	11122	AI B
Vogel	Südstr. 5	21.5.1984	343434	WI B
Reiter	Seegweg	13.10.1986	987654	TM B

AI-Studenten

Name	Adresse	Geb_Dat	MatrNr	Studiengang
Kuhn	Ostweg 2	18.10.1985	11122	AI B
Berger	Nordgasse	1.5.1984	22111	AI B

* **Vereinigung** T_1 und T_2 seien verträglich $\rightarrow$ $T_1 \cup T_2 = \{ r : r \in T_1 \vee r \in T_2 \}$

Studentinnen $\cup$ AI-Studenten

Name	Adresse	Geb_Dat	MatrNr	Studiengang
Kuhn	Ostweg 2	18.10.1985	11122	AI B
Vogel	Südstr. 5	21.5.1984	343434	WI B
Reiter	Seegweg	13.10.1986	987654	TM B
Berger	Nordgasse	1.5.1984	22111	AI B

* **Durchschnitt** T_1 und T_2 seien verträglich $\rightarrow$ $T_1 \cap T_2 = \{ r : r \in T_1 \wedge r \in T_2 \}$

T_1 und T_2 nicht verträglich $\rightarrow$ $T_1 \cap T_2 = \{ \}$

Studentinnen $\cap$ AI-Studenten

Name	Adresse	Geb_Dat	MatrNr	Studiengang
Kuhn	Ostweg 2	18.10.1985	11122	AI B

* **Differenz** $T_1 \setminus T_2 = \{ r : r \in T_1 \wedge r \notin T_2 \}$

Studentinnen \ AI-Studenten

Name	Adresse	Geb_Dat	MatrNr	Studiengang
Vogel	Südstr. 5	21.5.1984	343434	WI B
Reiter	Seegweg	13.10.1986	987654	TM B

* **Produktmenge** $T_1 \times T_2 = \{ r_1 r_2 : r_1 \in T_1 \wedge r_2 \in T_2 \}$ $(m_1, m_2, ..., m_n)(n_1, n_2, ..., n_n)$

AI-Studenten $\times$ Studiengaenge

Name	Adresse	Geb_Dat	MatrNr	Studiengang	Name	Abkürzung	Abschluss	Regelstudienzeit	Credits
Kuhn	Ostweg 2	18.10.1985	11122	AI B	Angewandte Informatik Bachelor	AI B	BSc	6 Semester	180
Kuhn	Ostweg 2	18.10.1985	11122	AI B	Wirtschaftsinformatik Bachelor	WI B	BSc	7 Semester	210
Kuhn	Ostweg 2	18.10.1985	11122	AI B	Technik-Management Bachelor	TM B	BSc	7 Semester	210
Kuhn	Ostweg 2	18.10.1985	11122	AI B	Angewandte Informatik Master	AI M	MSc	4 Semester	120
Berger	Nordgasse	1.5.1984	22111	AI B	Angewandte Informatik Bachelor	AI B	BSc	6 Semester	180
Berger	Nordgasse	1.5.1984	22111	AI B	Wirtschaftsinformatik Bachelor	WI B	BSc	7 Semester	210
Berger	Nordgasse	1.5.1984	22111	AI B	Technik-Management Bachelor	TM B	BSc	7 Semester	210
Berger	Nordgasse	1.5.1984	22111	AI B	Angewandte Informatik Master	AI M	MSc	4 Semester	120

Besser geeignet für die Kombination von Tabellen ist der „Join“ (Join = Anschluss, Verbund)

$$T_1 \subset M_1 \times ... \times M_n \times G_1 \times ... \times G_k \qquad T_2 \subset G_1 \times ... \times G_k \times M_{n+1} \times ... \times M_m$$

$$\text{Join}(T_1, T_2) = \{ r \in M_1 \times ... \times M_n \times G_1 \times ... \times G_k \times M_{n+1} \times ... \times M_m : \exists$$
$$s \in M_1 \times ... \times M_n, t \in G_1 \times ... \times G_k, u \in M_{n+1} \times ... \times M_m :$$
$$st \in T_1 \wedge tu \in T_2 \}$$

```
Join(AI-Studenten, Studiengaenge,
     Studiengang = Studiengaenge.Abkuerzung)
```

Name	Adresse	Geb_Dat	MatrNr	Studiengang	Name	Abkürzung	Abschluss	Regelstudienzeit	Credits
Kuhn	Ostweg 2	8.1.1985	11122	AI B	Angewandte Informatik Bachelor	AI B	BSc	6 Semester	180
Berger	Nordgasse	1.5.1984	22111	AI B	Angewandte Informatik Bachelor	AI B	BSc	6 Semester	180

*** verschiedene Arten von Join**

Equi Join : Verbund der Tabellen über Attribute, deren Inhalte übereinstimmen z.B.

```
Join (AI-Studenten, Studiengang,
      AI-Studenten.Studiengang=Studiengaenge.Abkuerzung)
```

Outer Join: Alle Datensätze der einen Tabelle z.B. von T_1, auch wenn es keine passenden in T_2 gibt, aber nur die Datensätze von T_2, die zu Datensätzen von T_1 passen.
Beispiel: Alle Studiengänge, auch wenn noch keine Studenten dazugehören

Theta Join : Verallgemeinerung von Join. Statt nur Gleichheitszeichen, sind hier beliebige Relationen verwendbar. ($>, <, \neq$, ...)

*** Projektion :** Auswahl von bestimmten Spalten einer Tabelle
z.B. `PROJECTION (Studentinnen, (Name, Adresse))`
Wählt die Spalten *Name* und *Adresse* aus der Tabelle Studentinnen aus

Name	Adresse	Geb_Dat	MatrNr	Studiengang
Kuhn	Ostweg 2	18.10.1985	11122	AI B
Vogel	Südstr. 5	21.5.1984	343434	WI B
Reiter	Seegweg	13.10.1986	987654	TM B

*** Selektion :** Auswahl von bestimmten Zeilen einer Tabelle, mittels eines Prädikats.
`SELECTION (T, C)` = $\{r : r \in T_1 \wedge c(r)\}$

z.B. `SELECTION (Studentinnen, Studiengang = "AI")`

Name	Adresse	Geb_Dat	MatrNr	Studiengang
Kuhn	Ostweg 2	18.10.1985	11122	AI B
Vogel	Südstr. 5	21.5.1984	343434	WI B
Reiter	Seegweg	13.10.1986	987654	TM B

*** Division :** Umkehrung des Produkts
`DIVISION (`$T_1 \times T_2$`, `T_2`) = `T_1
$T_1 \subset Y \times Z$, $T_2 \subset Z$ (Y und Z können mehrere Spalten darstellen)
`DIVISION (`T_1`, `T_2`)` ist eine Tabelle mit den Spalten Y. In ihr sind nur Datensätze, die in T_1 in Kombination mit <u>allen</u> Datensätzen von T_2 vorkommen.

Beispiel 1:

T1

S1	S2
a	1
b	1
c	1
a	2
b	2
c	2
a	3
c	3

T2

S2
1
2
3

T3

S1
a
c

```
T3 = DIVISION(T1, T2)
```

umständlich in SQL:

```
T3 = SELECT S1 FROM T1 T1Kopie1
     WHERE NOT EXISTS
       (SELECT * FROM T2
        WHERE S2 NOT IN
         (SELECT S2 FROM T1 T1Kopie2
          WHERE T1Kopie1.S1 = T1Kopie2.S1));
```

Beispiel 2: Gib die Matrikelnummer der Studenten aus, die im Wintersemester 2009/2010 an all den Kursen teilnehmen, an denen der Student mit der MatrNr 12345 teilnimmt!

1. Kurse von Nr 12345 suchen

```
T1 = PROJECTION (
        SELECTION (nimmt_teil, MatrNr= 12345 AND Semester = 20095),
        KursNr
     )
```

T_1 =

1438
1803

2. Division

```
DIVISION (
)
```

MatrNr	KursNr
12345	1438
12345	1803
11122	1803
11122	9999
343434	1803
343434	1111
987654	1803
987654	1111
987654	1438

Programme in der Relationsalgebra durch Kombination der Grundoperation

Insert :
```
Neu_Mit = {(Grohe, Abweg 5, 1.1.60, ...)}
Studenten = Studenten ∪Neu_Mit

ERG = PROJECTION (
        JOIN (
         Studenten, Studiengaenge,
         Studenten.Studiengang = Studiengaenge.Abkürzung
        ),
        Studenten.Name, Studiengaenge.Abschluss
      );
Print (ERG);
```

3.3.4 Datenmanipulation lesend: Abfragen mit SQL

```
SELECT <Spaltenname> FROM <Tabellennamen> WHERE <Bedingung>
```

In der „SELECT“ Anweisung sind Join, Selektion und Projektion der Relationenalgebra beinhaltet.

Beispiele:

- **Alle Daten** einer **Tabelle** anzeigen, z. B. alle Studiengänge

- **Selektion:** Alle Studenten des Studiengangs AI anzeigen

- **Projektion:** Namen und Telefonnummern aller Professoren anzeigen

- **Join:** Die Daten aller Studenten zusammen mit den Daten ihres Studiengangs anzeigen

- **Produkt** der Tabellen Studenten und Studiengang

- **Selektion, Projektion, Join gemischt:** Namen aller Studenten, die ihr Studium am 1.9.2008 begonnen haben und den Abschluss BSc austreben

- **Kopieren von Tabellen**

- **Kopieren von Datensätzen**

- **Mengen-Befehle**

UNION: Vereinigung zweier Tabellen

INTERSECT: Durchschnitt

EXCEPT (bei Oracle: MINUS)**: Mengendifferenz** (Beispiel analog zu oben)

- **Sortierung mit ORDER BY**

Beispiel: Sortiere Studenten nach Studiengang, innerhalb eines Studiengangs nach Studienbeginn und schließlich nach Name.

- **Statistische Funktionen: SUM, AVG, COUNT, MIN, MAX**

Beispiel: Berechnen Sie Summe, Durchschnitt, Maximum und Anzahl der Noten

- **GROUP BY:** Teile die Datensätze in Gruppen ein und erstelle Statistik für jede Gruppe

`Join (Studenten, nimmt_teil, Studenten.MatrNr = nimmt_teil.MatrNr)`

Name	Adresse	Geb_Dat	MatrNr	Studiengang	MatrNr	KursNr	Note	Versuch	Semester
Abel	Südstr. 1	7.11.1985	12345	TM B	12345	1438	2,0	1	WS 03/04
Abel	Südstr. 1	7.11.1985	12345	TM B	12345	1803	BE	1	WS 04/05
Berger	Nordgasse	1.5.1984	22111	AI B	22111	1803	BE	1	SS 04
Berger	Nordgasse	1.5.1984	22111	AI B	22111	1438	2,3	1	WS 04/05
Berger	Nordgasse	1.5.1984	22111	AI B	22111	9999	3,0	1	SS 03
Kuhn	Ostweg 2	18.10.1985	11122	AI B	11122	1803	BE	1	WS 04/05
Kuhn	Ostweg 2	18.10.1985	11122	AI B	11122	9999	2,0	1	SS 03
Reiter	Seegweg	13.10.1986	987654	TM B	987654	1803	BE	1	WS 04/05
Reiter	Seegweg	13.10.1986	987654	TM B	987654	1111	1,0	1	WS 04/05
Reiter	Seegweg	13.10.1986	987654	TM B	987654	1438	1,3	1	SS 04
Vogel	Südstr. 5	21.5.1984	343434	WI B	343434	1803	BE	1	WS 03/04
Vogel	Südstr. 5	21.5.1984	343434	WI B	343434	1111	3,0	1	SS 04

- **Unterabfragen**

Beispiel: Welche Studenten haben in DABS eine bessere Note als der Durchschnitt?

- **IN**: Ist Datensatz Element einer Tabelle?

Beispiel: Welche Studenten besuchen einen Kurs, der vom Studiengang AI angeboten wird?

- **(NOT) EXISTS**: Ist Tabelle leer oder nicht?

Beispiel: Welche Studenten haben noch nie an einer Vorlesung oder Prüfung teilgenommen?

3.3.5 VIEW

- ist **virtuelle** Tabelle: Teil oder Kombination von realen Tabellen
- ist **gespeicherte SQL-Abfrage**
- Verwendung bei SELECT wie Tabellen
- dient zum Aufbau **externer Sichten** (logische Datenunabhängigkeit)
- ermöglicht dedizierten Zugriffsschutz: Zugriffsrecht nur auf Views, nicht auf Tabellen
- Änderung der Tabelle ⇒ View ändert sich automatisch

Syntax:

```
CREATE VIEW <Name> AS
SELECT  <Feldliste> FROM <Tabellenliste> WHERE <Bedingung>;
```

Beispiel: Erzeuge VIEWs Studentinnen und AI-Studenten aus Studenten

Damit werden virtuelle Tabellen als Teile der Kombination von physikalischen Tabellen erzeugt. Bestimmte Benutzergruppen erhalten dann nur Zugriffsrechte für bestimmte VIEWs, nicht mehr für alle Tabellen.

3.4 Normalformen von (relationalen) Datenbanken

Ziele:

- Integrität (Konsistenz) der Daten
- Effizienz
  - Speicherplatz
  - Zugriffszeit
  - Programmentwicklung, Wartung, Erweiterbarkeit
- Übersichtlichkeit
- Datenschutz

3.4.1 Informelle Regeln für RDBS

Beispiel 1: Jedes Gebäude der FH habe seinen eigenen Hausmeister. Für schnellen Zugriff werden die Daten des Hausmeisters in der Tabelle Gebäude mitgespeichert: Gebäude (GNr, GBez, Standort, Art, HPersNr, HName, HAdresse, HTel, Hgehalt,...). Für die anderen Mitarbeiter gibt es die Tabelle Angestellte.

Beispiel 2: Die Vorlesungen werden zusammen mit den Daten der jeweilgen Dozenten gespeichert: DozKurs (PersNr, Name, Adresse, GebDat, Besoldung, KNr, KBez, KLArt)

Semantik der Tabelle „Angestellte“ bei Beispiel 1 ist schwer zu erklären:
Hier werden die Daten aller Angestellten gespeichert außer den Daten des Hausmeisters, die zusammen mit der Tabelle „Gebäude“ gespeichert werden.

Entwurfsregel 1:
Entwurf eines Schemas so, dass es einfach zu erklären ist. Jede Tabelle sollte nur eine Entity (Eintrag)
oder eine Relation repräsentieren.

Redundanz: Im Beispiel 2 müssen für jede Vorlesung eines Dozenten alle Daten des Dozenten gespeichert werden. Dadurch wird Platz verschwendet und es treten Anomalien auf. (Anomalie z.B. Hausmeister gehört nicht in die Tabelle Gebäude)

Anomalien werden häufig von Redundanzen erzeugt.

Einfügeanomalie: In Beispiel 2 kann keine Vorlesung eingefügt werden, bei der der Dozent noch nicht feststeht, es sei denn mit NULL-Werten bei den Dozentendaten. Steht der Dozent fest, so müssen alle Daten mit den Datensätzen für die anderen Vorlesungen dieses Dozenten übereinstimmen. Auch ein Dozent ohne Vorlesungen ist schwer einzufügen.

Löschanomalie: Wird bei Beispiel 1 ein Gebäude abgerissen und die Gebäudedaten gelöscht, so werden auch die Daten des Hausmeisters gelöscht.

Änderungsanomalie: Ändern sich die Daten eines Professors, so muss die Änderung bei allen zugehörigen Vorlesungssätzen in gleicher Weise erfolgen.

Entwurfregel 2:
Beim DB-Design sind Anomalien möglichst auszuschließen und Redundanzen zu vermeiden. Sind sie aus Effizienzgründen unvermeidlich, so sollen sie klar dokumentiert werden, damit Anwendungen möglichst sicher programmiert werden können.

Beispiel 3: Die Tabelle Angestellte bekommt ein zusätzliches Attribut E-Mail. Nur 10% der Angestellten haben eine E-Mail-Adresse.

Nullwerte haben verschiedene Bedeutung (siehe Beispiel 3):

- Attribut bedeutungslos für Datensatz (z.B. Reinigungskraft)
- E-Mail-Adresse für Angestellten noch nicht vorhanden
- Attribut vorhanden aber unbekannt
- Attribut bekannt aber noch nicht eingetragen

Platzverschwendung durch viele Nullwerte bei einem Attribut.

Entwurfregel 3:
Attribute mit möglichen Nullwerten vermeiden. Wenn nötig, so sollten Nullwerte nur bei wenigen Datensätzen auftreten.

Beispiel 4: Die Tabelle Stud_Kurs (MatrNr, Name, Adr, ..., Studiengang, KursNr, Bezeichnung, Kurzname, Prüfung, SWS, Credits, Note, Versuch, Semester) soll nach den Regeln 1 und 2 aufgeteilt werden. Es entstehen die Tabellen Studenten (MatrNr, Name, Adresse, ..., Studiengang), Kurse (KursNr, Bezeichnung, Kurzname, PrüfArt, SWS, Credits) und nimmt_teil (MatrNr, Kurzname, Note, Versuch, Semester)

Ursprüngliche Daten:

1234,Huber,88255 Bft Weg 1, ..., EI B, 1438,Mathematik1,MATH1,K120,8,10, NULL, 1, WS 03/04
9876, Meier, 88250 Wgt Str 2, ..., MB B, 456, Mathematik1, MATH1, K90, 6, 8, 4.0, 2, SS 04
Aufgeteilte Daten :

Studenten	Kurse	nimmt_teil
1234, Huber, ..., EI B	1438, Mathematik1, MATH1, K120, 8, 10	1234, MATH1, NULL, 1, WS 03/04
9876, Meier, ..., MB B	456, Mathematik1, MATH1, K90, 6, 8	9876, MATH1, 4.0, 2, SS 04

Aufgeteilte Daten wieder zusammenfügen:

```
SELECT * FROM Studenten, Kurse, nimmt_teil
WHERE Studenten.MatrNr = nimmt_teil.MatrNr
  AND Kurse. Kurzname = nimmt_teil.Kurzname ;
```

Matr Nr	Name	Adr	...	Stud-gang	Kurs Nr	Bezeichnung	Kurz name	Prüf Art	SWS	Credits	Note	Ver-such	Semes-ter

Falsches Joinergebnis bei * (Datensatz Nr 2 und 3)

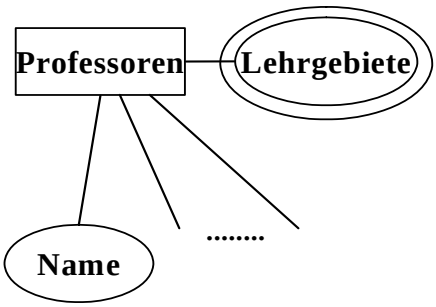
Entwurfregel 4:
Die Aufteilung in Tabellen sollte so erfolgen, dass beim Join über Fremdschlüssel keine unsinnigen Datensätze
entstehen. Verwende als Fremdschlüssel nur *eindeutige* Attribute der anderen Tabelle, z.B. den Primärschlüssel. (*Lossless Join Eigenschaft*)

3.4.2 Erste Normalform

Definition 1. Normalform:
Es gibt keine mehrwertigen Attribute.

Definition:
Ein mehrwertiges Attribut ist ein Attribut, das für einen Datensatz nicht nur einen Wert, sondern eine Menge von Werten speichern kann.

Beispiel: Ein Dozent hat verschiedene Lehrgebiete



in Datenbank

1234 | Richler | Seestr. 3 | 05.05.60 | ... | Mathematik, Robotik
nicht in 1. Normalform

MöglicheLösung : Eigene Tabelle ProfessorenLehrgebiete

PersNr	Lehrgebiet
1234	Mathematik
1234	Robotik
9999	Künstliche Intelligenz
...	

Überführen in 1. Normalform

1. Lösung: Datensatz wird für jedes Element des mehrwertigen Attributes wiederholt ⇒ Redunanz
2. Lösung: Breite Spalte für mehrwertiges Attribut, Menge als String eintragen und durch Trennzeichen trennen. ⇒ aufwändige Suche, Speicherplatzverschwendung
3. Lösung: Eigene Tabelle für mehrwertiges Attribut. Zuordnung zum Datensatz durch den Primärschlüssel der Ursprungstabelle.
   Bsp: Professoren (PersNr, Name, Adr, Geb_Dat, ...)
   ProfessorenLehrgebiete (PersNr, Lehrgebiet)
   Wie Lösung Nr. 1, aber ohne unnötige Redunanz
4. Lösung: Wenn die Anzahl der Elemente für das mehrwertige Attribut bei allen Datensätzen gleich ist, werden entsprechend viele Spalten definiert.
   Bsp: Professoren (PersNr, Name, Adr, GebDat, Lehrgebiet1, Lehrgebiet2..)
   ⇒ evtl. aufwendige Suche und Verarbeitung und evtl. Nullwerte

Aufgabe: Suchen Sie die Namen aller Professoren mit den Lehrgebieten Mathematik und Nachrichtentechnik. Geben Sie dazu für alle vier Lösungen die entsprechende SQL-Anweisung zur Suche an. Bei der vierten Lösung gehen Sie von drei Spalten für die Lehrgebiete aus.

3.4.3 Funktionelle Abhängigkeiten

Definition: Eine Tabelle (=Relation) habe die Attribute A, B, C, ... X, Y, Z seien Teilmengen der Attribute, z.B. X = {B,D}. **Y** heißt **funktionell abhängig von X**, in Zeichen $X \rightarrow Y$, wenn für zwei Zeilen der Tabelle, bei denen die Werte bei den Attributen von X übereinstimmen, auch die Werte von Y übereinstimmen.

Dies stellt eine Verallgemeinerung des Schlüsselbegriffs dar.

Beispiel: *StudKurs* : $\underbrace{\{\text{MNr,KNr}\}}_{X} \rightarrow \underbrace{\{\text{Versuch, Note, Semester}\}}_{Y}$

Beispiel:

A	B	C	D	E
a	1	x	1	I
c	2	x	1	II
b	1	z	2	III
a	1	y	1	IV
b	1	z	2	V

Welche der folgenden Abhängigkeiten gelten?
$\{B, D\} \rightarrow A$?
$E \rightarrow \{A, B, C, D\}$?
$\{C, D\} \rightarrow A$?

Aus einer Menge von Basisabhängigkeiten können weitere gewonnen werden:

Reflexivregel : $Y \subset X \Rightarrow X \rightarrow Y$
Erweiterung : $X \rightarrow Y \Rightarrow XZ \rightarrow YZ$
Transitivregel : $X \rightarrow Y, Y \rightarrow Z \Rightarrow X \rightarrow Z$
Aufteilung : $X \rightarrow YZ \Rightarrow X \rightarrow Y$
Vereinigung : $X \rightarrow Y, X \rightarrow Z \Rightarrow X \rightarrow YZ$
Pseudotransitivgesetz : $X \rightarrow Y, WY \rightarrow Z \Rightarrow WX \rightarrow Z$

Um alle funktionellen Abhängigkeiten beschreiben zu können, benötigt man eine minimale Menge von Basisabhängigkeiten und man bildet die Hülle bezüglich der sechs Regeln.
Die funktionellen Abhängigkeiten steuern einen Algorithmus, der aus einem gegebenen DB-Schema ein normalisiertes DB-Schema erzeugt.

Beispiel: Tabelle Ausleihe in einer Bibliothek

3.4.4 Zweite Normalform

Ursache für Redundanz: Es gibt verschiedene Datensätze, bei denen **Teilrelationen gleich** sind, z.B. haben verschiedene Ausleihvorgänge eines Kunden gleichen Namen und gleiche Adresse.

Kennzeichen ist die FA

Vermeide Redundanz durch **Zerteilen** der Tabelle

Definition Zweite Normalform
Ein Relationsschema ist in der 2. Normalform, wenn es in der 1. Normalform ist und es **keine FAn gibt**, bei denen

In der 2. Normalform im zerteilten Beispiel gibt es immer noch Redundanz: Wenn ein Buch in mehreren Exemplaren vorhanden ist, dann wird der Titel jedes Mal wiederholt.

Ursache ist die FA

3.4.5 Boyce-Codd-Normalform

Definition Boyce-Codd-Normalform (BCNF)
In der BCNF dürfen bei allen funktionellen Abhängigkeiten, bei allen Tabellen auf der linken Seite nur Superschlüssel stehen.

Algorithmus zur Umwandlung von der 1. Normalform in die BCNF

1) Trage bei allen Tabellen die funktionellen Abhängigkeiten ein. (Vollständiger Satz von Basisabhängigkeiten, aus denen sich alle Abhängigkeiten herleiten lassen).
2) Greife eine funktionelle Abhängigkeit heraus, die auf der linken Seite keinen Superschlüssel hat. $T : \{A_1,...,A_n\} \rightarrow \{B_1,...,B_m\}$
3) Zerteile Tabelle T in zwei Tabellen T_1 und T_2. T_1 hat alle Attribute von T außer $\{B_1,...,B_m\}$. T_2 hat Attribute $T : \{A_1,...,A_n,B_1,...,B_m\}$
4) Trage in T_1 und T_2 die funktionellen Abhängigkeiten ein.
5) Gibt es noch funktionelle Abhängigkeiten, die auf der linken Seite keine Superschlüssel haben? Ja : weiter bei 2, Nein : fertig.

Beispiel : Stud_Kurs

Matr Nr	Name	Adr	...	Studgang	Kurs Nr	Bezeich-nung	Kurz name	PrüfArt	SWS	Credits	Note	Ver-such	Semester

Graphische Notation von funktionellen Abhängigkeiten.

z.B. FA 3: {MatrNr, KursNr} → { Note, Versuch, Semester}

Gibt es funktionelle Abhängigkeiten, die auf der linken Seite keinen Superschlüssel haben?
Zerteile Stud_Kurs gemäß (3), in :

und

Gibt es funktionelle Abhängigkeiten, die auf der linken Seite keinen Superschlüssel haben?
Zerteile T_1 bezüglich funktioneller Abhängigkeit „2“ gemäß (3) in:

und

Ende, da alle Tabellen mit ihren funktionellen Abhängigkeiten in BCNF.

Der Algorithmus hat die **Lossles-Join-Eigenschaft**.

Er ist nicht **dependency-preserving**, d.h. es können funktionelle Abhängigkeiten der Ausgangstabelle in den Ergebnistabellen verschwunden sein.

Z.B. gilt Studkurs {Stdg, KBez} → {KNr}
Diese funktionelle Abhängigkeit taucht in dem zerteilten Schema nicht mehr auf.

Aufgabe: Bringen Sie die Tabelle Studiengang_Kurs in Boyce-Codd-Normalform.

Studiengang_Kurs

StgName	StgAb-kürzung	Abschluss	Regelstu-dienzeit	StgCredits	Kurs Nr	Bezei-chnung	Kurs-Kurzname	PrüfArt	SWS	Credits

Weitere Normalformen: Es gibt noch weitere Normalformen: die **fünfte** und die **Domain-Key-Normalform**. Diese spielen in der Praxis keine Rolle. Nähere Informationen siehe Literatur, z. B. Ramez A. Elmasri, Shamkant B. Navathe: „Grundlagen von Datenbanksystemen“, Kap.14 und 15.

3.4.6 Vierte Normalform

Wenn es in einem Relationsschema zwei oder mehr mehrwertige Attribute gilt und diese mit Redundanz in einer einzigen Tabelle in der 1. Normalform gespeichert werden, entsteht eine **Mehrwertige Abhängigkeit**.

Beispiel: Tabelle Professoren

PersNr	Name	…	TelListe	Lehrgebiete

PersNr	Name	…	Tel	Lehrgebiet

Hier besteht eine mehrwertige Abhängigkeit

Definition Mehrwertige Abhängigkeiten

Allgemein besteht in einer Relation R = *XYZ* die **Mehrwertige Abhängigkeit**
wenn für jeden Wert *x* bei *X* gilt: Kommt *x* in R mit *y* bei *Y* vor und *x* in R mit *z* bei *Z*, dann kommt *x* in R in der Kombination *xyz* vor.

R =

X	Y	Z
x	y	b
x	a	z
x	y	z
x	a	b

Definition Vierte Normalform

Ein Relationenschema ist in der **4. Normalform**, wenn es in Boyce-Codd Normalform ist und wenn es keine mehrwertigen Abhängigkeiten gibt.

Umformung:

Beispiel

PersNr	Name	Tel

PersNr	Name	Lehrgebiet

4 Datenbank-Programmierung

4.1 Eingebettetes SQL

Sprachumfang von SQL reicht nicht aus, es fehlt:

- Fallunterscheidung
- Schleifen
- Funktionsaufrufe
- rekursive Programmierung

Lösungen

- SQL erweitern
  - Funktionsbibliotheken zur Kommunikation mit DB
  - Application Program Interface (API)
  - Call Level Interface (CLI)
  - Im Programm Aufruf der API-Funktionen
  - Einbinden der Bibliothek
  - Einige APIs: OCI, ODBC, JDBC
- SQL in Host-Programmiersprache integrieren
  - embedded SQL
  - SQLJ

Client-Server Prinzip

Embedded SQL

Prinzipieller Ablauf

- C-Programm mit SQL
  - Precompiler
- C-Programm mit Aufruf von DB-Kommunikationsfunktionen
  - Compiler, Linker
- Ausführbares Client-Programm

UWA = User Work Area: Programmvariable, die Daten aus der DB aufnehmen.

Problem : SQL Liefert als Ergebnis Tabellen

Es ist nicht praktisch, ganze Tabellen in die UWA zu übernehmen, da damit sehr viel Arbeitsspeicher belegt würde, die Verarbeitung im Programm aber doch datensatzweise erfolgen müsste, da es keine Sprachmittel für Tabellenoperationen in einer klassischen Programmiersprache gibt.

Daher wird ein Zeiger verwendet, der auf die aktuelle Tabellenzeile zeigt, ein **Cursor**, und diese Zeile wird in die UWA übernommen. Alle Datensätze der Tabelle werden dann in einer Schleife bearbeitet.

Man geht daher sinnvollerweise so vor, dass Tabellenoperationen komplett in SQL ausgeführt werden und nur die Operationen in der Programmiersprache programmiert werden, die in SQL nicht zu bewältigen sind. Außerdem kann man dadurch eine Arbeitsentlastung des Servers erreichen: Alle Operationen in SQL werden auf dem Server durchgeführt, alle anderen Operationen vom Client. Zusätzlich sollte auf eine geringe Netzbelastung geachtet werden.

Beispiel: C-Programm mit Embedded SQL für die formatierte Notenausgabe bei Studenten

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

EXEC SQL BEGIN DECLARE SECTION;
EXEC SQL END DECLARE SECTION;
EXEC SQL INCLUDE sqlca; /*SQL communication area einbinden für Status und
Fehlermeldung */

int Anmeldung (); /* Logon in das ORACLE-Datenbanksystem */
int druckPersonendaten (int);
void druckNotenliste (int, int);

main ()
{
 char input[10];
 int mnr;
 int startsemester = 0;

 if (Anmeldung ()) { /* if1 */
  do {
   printf("\n\nGeben Sie die Matrikelnr eines Studenten ein, 0 fuer Abbruch:
");
   gets(input);
   mnr = atoi(input);

   if (mnr) startsemester = druckPersonendaten(mnr);
   else startsemester = 0;

   if (startsemester > 0) druckNotenliste(mnr, startsemester);
  } while (startsemester > 0);
 } /* end if1 */
}

int Anmeldung () /* Logon in das ORACLE-Datenbanksystem */
{
 EXEC SQL BEGIN DECLARE SECTION;
      VARCHAR user[16], pwd[10]; /* UWA Benutzername und Passwort */
 EXEC SQL END DECLARE SECTION;

 /* Benutzer nach Name und Passwort fragen */
 printf("Benutzername und DB-Verbindung: ");
 gets(user.arr);
 user.len = strlen (user.arr);
 printf("Passwort: ");
 gets(pwd.arr);
 pwd.len = strlen (pwd.arr);
 EXEC SQL CONNECT :user IDENTIFIED BY :pwd; /* Verbindung aufbauen */

 if (sqlca.sqlcode == 0) {
   printf("Verbunden mit ORACLE als %s\n\n", user.arr);
```

```
   return (1);
 }
 else {
  printf("\n\n FEHLER bei Anmeldung % .70s \n\n", sqlca.sqlerrm.sqlerrmc);
  return (0);
 }
}

/* Holt Name und Semester des angeforderten Studenten aus der DB */
int druckPersonendaten (int mnr) {
 EXEC SQL BEGIN DECLARE SECTION; /* UWA */
      VARCHAR sname[50];
      char sBegJahr[5];
      char sBegMonat[3];
      int matrNr = mnr;
 EXEC SQL END DECLARE SECTION;

 int startsemester = 0;

 EXEC SQL WHENEVER SQLERROR GOTO sqlerror; /* unspezifische Fehlerbehandlung */

 EXEC SQL DECLARE Student_cursor CURSOR FOR /* Cursor für Studentendaten */
      SELECT Name, to_char(Studienbeginn, 'yyyy') StdBegJahr,
to_char(Studienbeginn, 'mm') StdBegMonat
      FROM Studenten WHERE MatrNr = :matrNr;

 EXEC SQL OPEN Student_cursor; /* Abfrage ausführen */

 /* ersten Datensatz holen */
 EXEC SQL FETCH Student_cursor INTO :sname, :sBegJahr, :sBegMonat;

 if (sqlca.sqlcode == 0) { /* Wenn Student gefunden */
  sname.arr[sname.len] = '\0';
  printf ("Notenausdruck fuer %s, %i\n\n", sname.arr, mnr);

  /* Startsemester bestimmen */
  if (atoi(sBegMonat) < 6) startsemester = atoi(sBegJahr) * 10;
  else startsemester = atoi(sBegJahr) * 10 + 5;
 }
 else printf("Keine gültige Matrikelnummer!");

 EXEC SQL CLOSE Student_cursor;
 return (startsemester);

 sqlerror:  /* unspezifische Fehlerbehandlung */
  printf("\n\n FEHLER in ora_in % .70s \n\n", sqlca.sqlerrm.sqlerrmc);
  EXEC SQL WHENEVER SQLERROR CONTINUE;
  return(0);
}

/* Funktion druckt die Notenliste für den Studenten mit Matrikelnummer MNr,
** kein Rückgabewert*/
void druckNotenliste(int mnr, int startsemester) {
  /* Noten auslesen */
 EXEC SQL BEGIN DECLARE SECTION; /* UWA */
      VARCHAR kname[6];
      int note;
      int versuch;
      int semester;
      int matrNr = mnr;
      /* Indikatorvariablen zum Abfangen von NULL-Werten: */
      short k_i, v_i, s_i;
 EXEC SQL END DECLARE SECTION;

 int sem; /* Zähler für Semester */
 EXEC SQL DECLARE Noten_cursor CURSOR FOR
 /* Cursor für alle Noten eines Studenten */
   SELECT Kurzname, Note, Versuch, Semester
   FROM nimmt_teil, Kurse
   WHERE nimmt_teil.KursNr = Kurse.KursNr
      AND MatrNr = :matrNr
      AND Note is not null
```

```
   ORDER BY StdSem, Kurse.KursNr, Versuch;

 EXEC SQL OPEN Noten_cursor; /* Abfrage ausführen */
 if (sqlca.sqlcode < 0) { /* spezifische Fehlerbehandlung */
   printf("Fehler bei Notenabfrage\n");
   printf("% .70s \n\n", sqlca.sqlerrm.sqlerrmc);
   return;
 }

 /* Hole erste Note des Studenten in UWA */
 EXEC SQL FETCH Noten_cursor INTO :kname:k_i, :note, :versuch:v_i,
:semester:s_i;
 if (sqlca.sqlcode < 0) { /* spezifische Fehlerbehandlung */
   printf("Fehler beim Holen der ersten Note\n");
   printf("% .70s \n\n", sqlca.sqlerrm.sqlerrmc);
   return;
 }
 if (k_i == 0) kname.arr[kname.len] = '\0'; /* Stringende setzen */

 /* Tabelle mit Noten drucken */
 /* Drucke Spaltenüberschriften: "Fach" und Semester für 7 Semester ab
Startsemester */
 printf ("Fach\\Semester\t");
 for (sem = startsemester; sem < startsemester + 40; sem = sem + 5) {
  /* Wandle Semesternummern wie 20065 in lesbare Semesterangaben um: W 2006/7
*/
  if ((sem % 10) == 0) { /* Sommersemester */
   semester = sem / 10;
   printf("S %d       \t", semester);
  }
  else { /* Wintersemester */
   semester = (sem - 5) / 10;
   printf("W %d/%d \t", semester, (semester % 10) + 1);
  }
 }

 /* Noten drucken */
 while (sqlca.sqlcode == 0) {
   /* Dieser Programmabschnitt funktioniert nur, wenn es zu einem zweiten oder
      dritten Versuch auch einen ersten Versuch gibt; sonst wird eine falsche
      Tabellenstruktur ausgedruckt. */
   if ((k_i != 0) || (v_i != 0) || (s_i != 0))
     printf("\nFehler: Noteneintrag unvollstaendig!\n");
   else {
     if (versuch == 1) {
       printf("\n%s\t\t", kname.arr);
       for (sem=startsemester; sem < semester; sem = sem + 5) printf("\t\t");
     }
     else for (sem=sem+5; sem < semester; sem = sem + 5) printf("\t\t");

     printf("%2.1f (%d)\t", note/10.0, versuch); /* gespeichert ist Note*10 */
   }

   /* Hole nächste Note des Studenten in UWA */
 EXEC SQL FETCH Noten_cursor INTO :kname:k_i, :note, :versuch:v_i,
:semester:s_i;
 if (k_i == 0) kname.arr[kname.len] = '\0'; /* Stringende setzen */
 } /* end while */
 if (sqlca.sqlcode < 0) { /* spezifische Fehlerbehandlung */
   printf("Fehler beim Holen einer Note\n");
   printf("% .70s \n\n", sqlca.sqlerrm.sqlerrmc);
 }

 EXEC SQL CLOSE Noten_cursor; /* Speicherplatz für Abfrage freigeben */
}
```

Typische Konstrukte

- UWA vereinbaren

```
EXEC SQL BEGIN DECLARE SECTION;
<C-Variablen-Deklaration>
```

```
EXEC SQL END DECLARE SECTION;
```

- Indikatorvariablen
  Indikator, ob Feldwert NULL ist. Datentyp `short`

- Communication Area
  ```
  EXEC SQL INCLUDE sqlca;
  ```

- Anmelden an DB
  ```
  EXEC SQL CONNECT <user> IDENTIFIED BY <pwd>;
  ```

- Cursor (Abfrage) deklarieren
  ```
  EXEC SQL DECLARE <cursorname> CURSOR FOR
  <SELECT-Anweisung>
  [FOR UPDATE OF <Spalte>];
  ```

- Cursor öffnen
  ```
  EXEC SQL OPEN <cursor>;
  ```

- Datensatz holen
  ```
  EXEC SQL FETCH <cursorname> INTO <Variablenliste>;
  ```

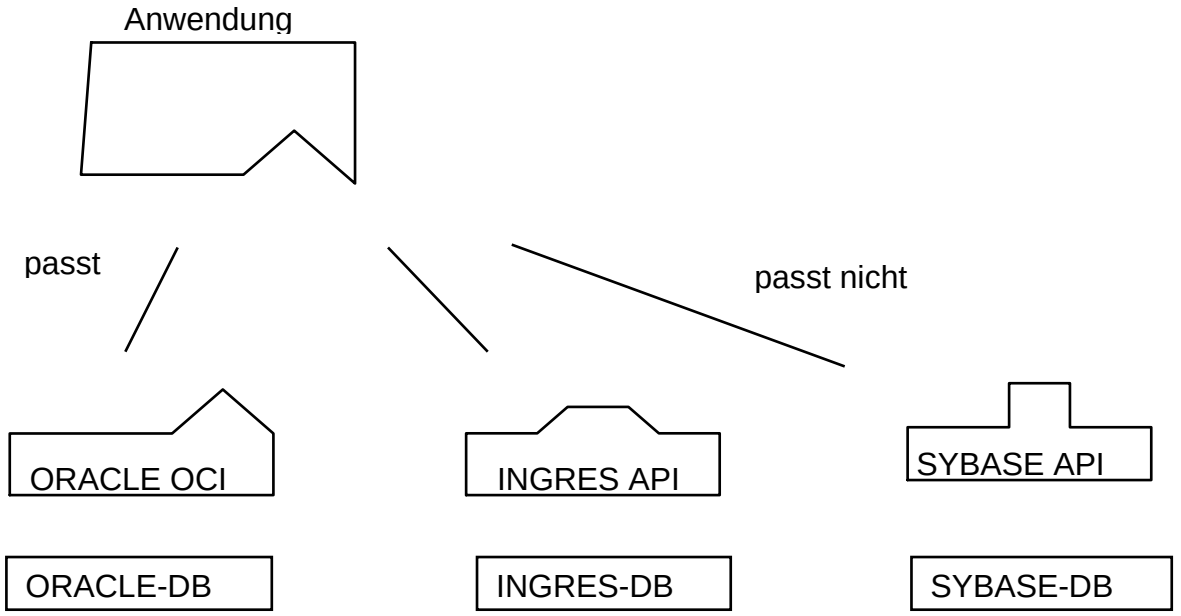
- Update bei Cursor
  ```
  UPDATE ... WHERE CURRENT OF <cursorname>;
  ```

- Allgemeine Fehlerbehandlung
  ```
  EXEC SQL WHENEVER SQLERROR GOTO <Label>;
  ```

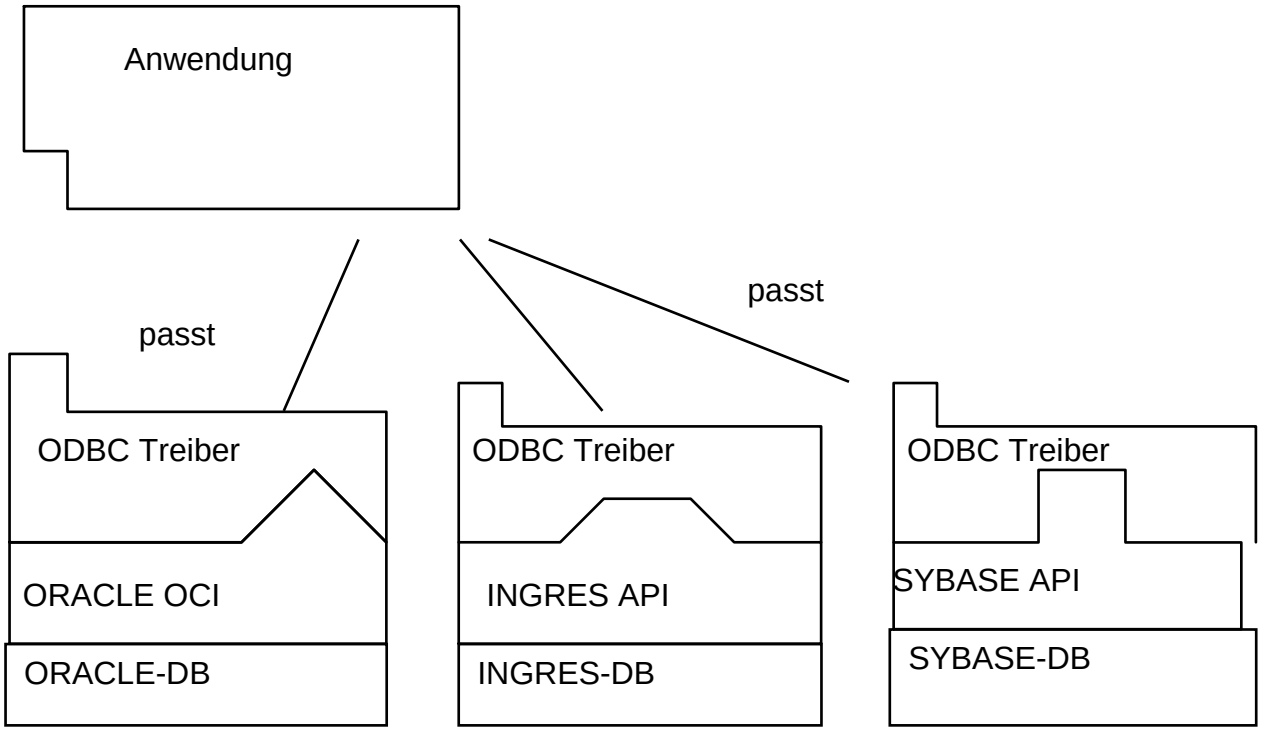
- Transaktion abschließen
  ```
  EXEC SQL COMMIT WORK RELEASE;
  EXEC SQL ROLLBACK WORK RELEASE;
  ```

4.2 ODBC = Open DataBase Connectivity

Die Arbeit mit proprietären APIs ist zwar effizient, aber immer nur für ein DBMS passsend.



ODBC stellt für alle DBMS eine einheitliche API bereit:



Wissenswertes zu ODBC

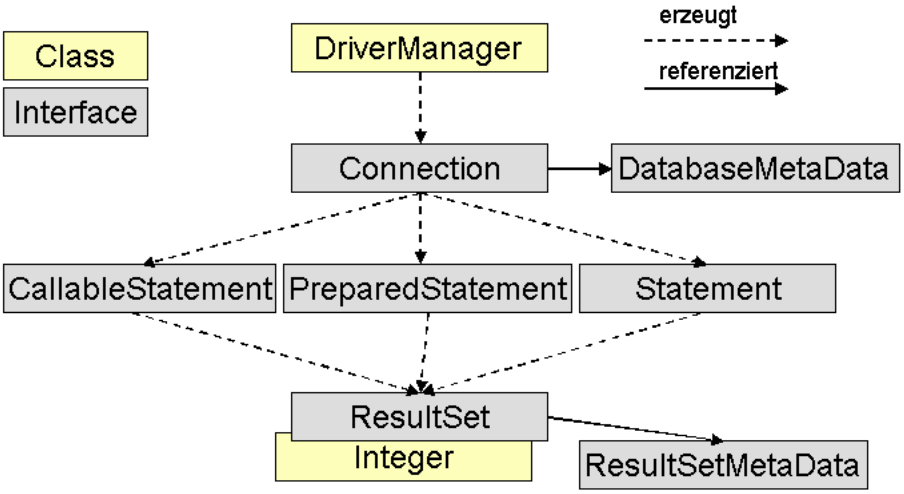
- Initiator: Microsoft und andere
- Zweck: Einheitliche API für verschiedene DBMS
  (API = Application Program Interface)
- CLI = Call Level Interface
  Implementierung der Definition der X/Open Group
- Sammlung von C-Funktionen
- Grundlage: Client-Server Architektur
- Komponenten von ODBC:
  - Treiber-Manager
  - je ein Treiber für jedes DBMS
- Da ODBC relativ kompliziert ist, gibt es viele APIs, die auf ODBC aufsetzen aber einfacher sind, z. B. ADO für Visual Basic oder C++.

4.3 JDBC = Java Database Conectivity

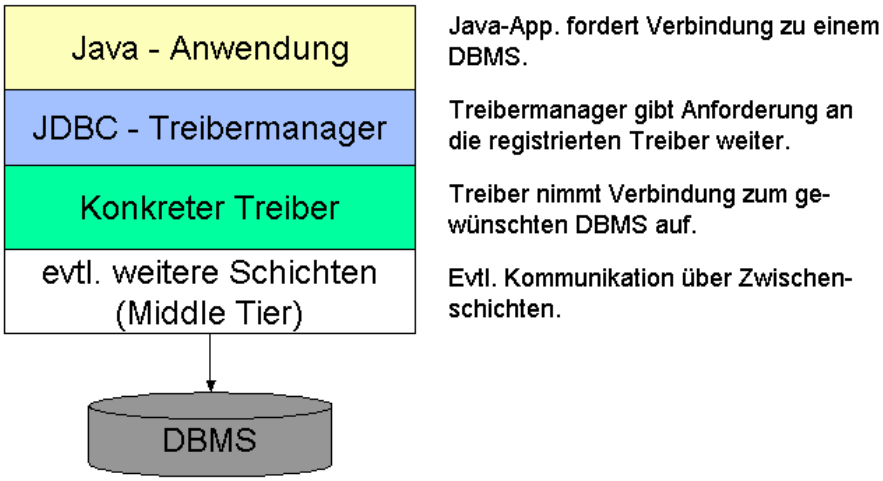
Was ist JDBC?

- API für Datenbankzugriff
- Speziell für die Programmiersprache JAVA
- Wie ODBC
  - Call Level Interface (CLI)
  - Basiert auf X/Open-Standard
  - Unabhängig vom DBMS
  - Treiber DBMS-spezifisch
- Unterschiede zu ODBC
  - Methoden statt Funktionen
  - Objekte statt Handle

JDBC-Klassen/Schnittstellen-Übersicht (unvollständig)



Treiber in JDBC

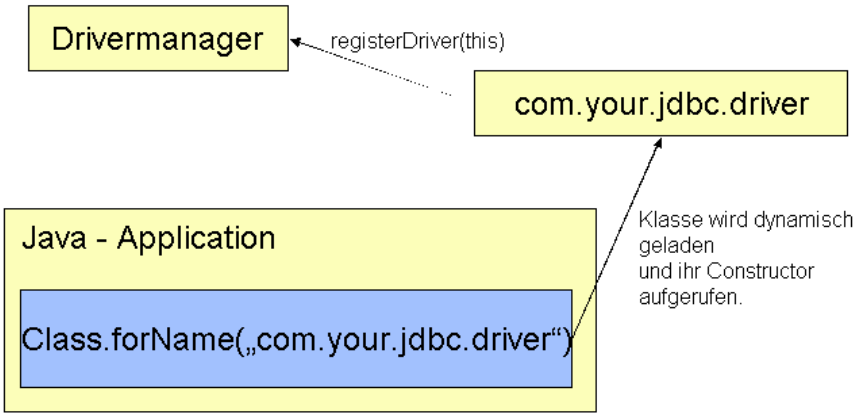


JDBC Funktionsweise

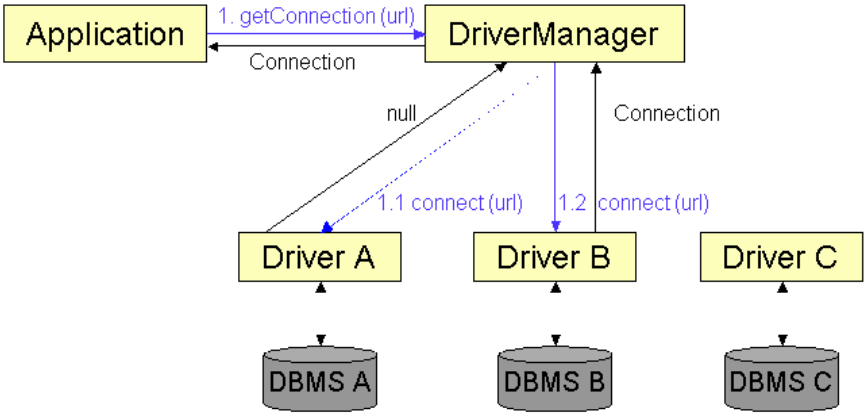
- JDBC spezifiziert nur Schnittstellen
- Implementation durch DBMS-Hersteller: Treiber
- Es gibt 4 Treibertypen
    - Typ1: JDBC-Treiber stützt sich auf ODBC, JDBC-ODBC Bridge
    - Typ2: JDBC-Treiber stützt sich auf proprietären Treiber, z. B. OCI
    - Typ3: JDBC-Treiber stützt sich auf serverseitigen Middleware-Treiber
    - Typ4: JDBC-Treiber kommuniziert direkt mit der Datenbank
- Typ 3 und 4 sind vorzuziehen: pure JAVA
- Treibermanager lädt Treiber
- Treiber stellt Verbindung zur Datenbank her

Laden des JDBC-Treibers

Da in der Applikation nicht ein bestimmter Treiber festgelegt werden muss, wird die Treiberklasse dynamisch zur Laufzeit mit "Class.forName" geladen und instanziiert.



Verbindung zur Datenbank: Connection Objekt erzeugen



Programm mit JDBC: Typischer Ablauf als Programmcode

```
// Treiber laden
Class.forName("packageName.YourDriver");

// Verbindung zum DBMS aufbauen
Connection c =
DriverManager.getConnection("yourURL","user","pwd");

// Statement erzeugen
Statement stmt = con.createStatement("SELECT * FROM yourTable");

// Statement ausführen
ResultSet rs = stmt.executeQuery();

// Ergebnismenge bearbeiten
while (rs.next()) { // Cursor eine Zeile weiter bewegen
     System.out.println(rs.getInt("emp_id") + rs.getString(2));}

// Aufräumarbeiten
rs.close();    // Ergebnismenge schließen
stmt.close();  // Statement schließen
con.close();   // Verbindung zum DBMS schließen
```

Die JDBC-Klassen: Connection, DatabaseMetaData

- Ein Objekt vom Typ Connection repräsentiert eine physikalische Verbindung zum DBMS
- Connection steuert Transaktionen mit DBMS

- Objekte vom Typ DatabaseMetaData enthalten Informationen über Struktur der Datenbank: Tabellen, Attribute, Domänen, Stored Procedures
- Datenbank-Metadaten sind von DBMS zu DBMS unterschiedlich organisiert.
  Hier zahlt sich die standardisierte Schnittstelle JDBC aus

Statements

- Statements sind die Arbeitspferde in JDBC
- Mit ihnen werden Daten manipuliert und gesichtet
- Es gibt 3 Typen mit verschiedenen Aufgaben
- Statement: zur Datenmanipulation und zum Erzeugen von ResultSets.
  `Statement stmt = connection.createStatement()`
- PreparedStatement = Precompiled Statement
  ```
  PreparedStatement pstmt = connnection.prepareSatement(
   "INSERT INTO emp VALUES (?,?,?,?,?);");
  ```
- CallableStatement: zum Ausführen von Stored Procedures
- Statement kann nicht mit new erzeugt werden.
- Es wird bei einem Connection Objekt angefordert:
  Dadurch bleibt der Bezug zur Connection bestehen

ResultSet

- ResultSets enthalten die Ergebnis-Menge eines ausgeführten Statements
- Wird von einem Statement erzeugt
- Beispiel:
  `ResultSet rs = stmt.executeQuery( "SELECT * FROM employee;")`
- Jedes RS hat einen Cursor der auf die aktuelle Zeile der Ergebnismenge zeigt.
- Verschiedene ResultSet-Typen (je nach Treiber und Anforderung durch Applikation)
  - Forward only
  - Scroll-insensitive
  - Scroll-sensitive
  - updateable, usw
- Zugriff auf Spaltenwerte mit
  - getXXXX (Spaltenname) oder
  - getXXXX(Spaltennummer)
  - XXXX steht für den Datentyp

Cursor

emp_id	emp_lname	Emp_fname	salary
11	Ryan	Jack	100.000
23	Scooter	John	200.000
56	Kemper	Andreas	200.000
78	Schmitz	Fredi	80.000
89	Kohl	Helmut	200.000
102	Clinton	George	250.000

Beispiel:

```
rs.getInt("emp_id")     => 56
rs.getString(2)         => "Kemper"
...
```

Batch-Update

- Standard: Neues Statement-Objekt für jede Datenmanipulation (insert, update, delete)
- Schneller: Mehrere Aufträge sammeln, dann gemeinsam ausführen: Batch-Update
- Beispiel

```
stmt.addBatch ("insert ...")
stmt.addBatch ("insert ...")
...
Int [] UpdateCounts = stmt.executeBatch ()
```

Erweiterungen: JDBC Standard Extension API

- Kern von JDBC soll einfach und klein sein
  Standard-Erweiterungen für JDBC werden in die Standard Extension API ausgelagert.
- 3 wichtige Standard-Erweiterungen für JDBC
  - Java Naming and Directory Interface: Verwendung von Datenquellen: DataSource
  - Connection Pooling: Wiederverwendung von Verbindungen spart Zeit
  - Distributed Transactions: Transaktionsmanagement bei mehreren Datenbanken

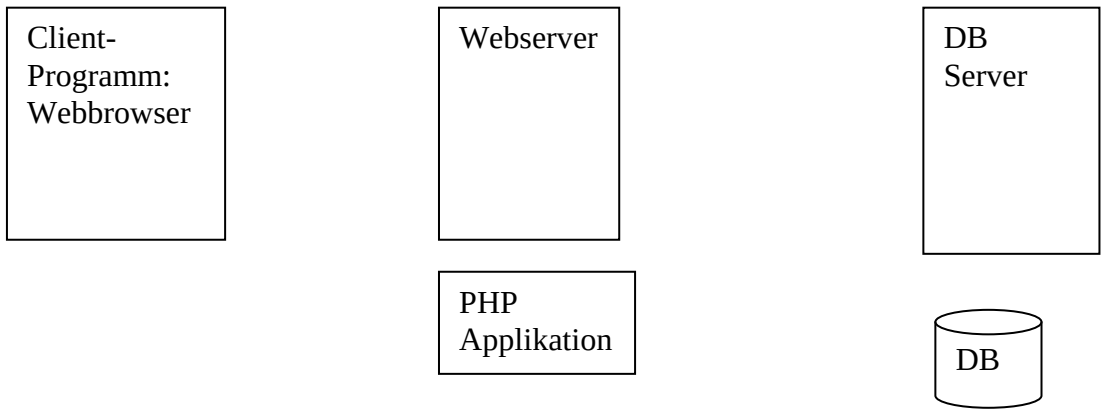
Beispiel: JAVA-Programm mit JDBC für die formatierte Notenausgabe bei Studenten

Das Beispielprogramm wird in der Vorlesung gezeigt. Der Source-Code ist im Moodle-Kurs als Download verfügbar.

4.4 Web-Programmierung mit PHP

- PHP = Php Hypertext Preprocessor
- Script-Sprache
- Eingebettet in HTML
- Sprache wird interpretiert, nicht übersetzt
- 3-tier Architektur
- Ausführung auf dem Webserver
- Besonders für Kommunikation mit Datenbanken
- Viele Spezialpakete, z. B. Grafik
- Open Source
- Häufig in Verbindung mit Open Source DB MySQL

3-tier Architektur im Web



Beispielprogramm: Notenliste drucken mit PHP

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN">
<html>
<head><title>NotenListeFormular</title></head>
<body>
<form name="form1" method=GET action="NotenListe.php">
```

```
  <label>Bitte geben Sie Ihre Matrikelnummer ein!</label>
  <p>Matrikelnummer: <input name=matrikelnummer type=text size=5></p>
  <p> <input type="submit" name="Submit" value="Notenliste anfordern"></p>
</form>
</body>
</html>
```

```
<html>
<head><title>NotenListe</title>
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-1">
<!-- PHP-Handler für NotenListeFormular.html -->
</head>
<body>
<?php
include("NotenFunktionen.php");

// Konstanten für Datenbankverbindung
define ("USER", "dabs_01");
define ("PASSWORD","dabs_01");
define ("SERVER_DATABASE", "//neptun.fbe.fh-weingarten.de:1521/namib");
// Connecting, selecting database
$link = oci_connect(USER, PASSWORD, SERVER_DATABASE) or exit('Could not connect: ' . oci_error());
echo "Connected successfully<br>\n";

$matrikelnummer = $_GET['matrikelnummer']; // Hole die Matrikelnummer aus dem aufrufenden Formular

$startsemester = druck_Personendaten($matrikelnummer, $link);

if ($startsemester) {druck_Notenliste($matrikelnummer, $startsemester, $link);}

oci_close($link); // Closing connection
?>
</body>
</html>
```

```
<!—File Notenfunktionen.php mit php-Funktionen für NotenListe.php -->
<?php
function druck_Personendaten($matrikelnummer, $dbcon) {
// Druckt eine Überschrift mit Personendaten
// Rückgabewert: Startsemester des Studenten in der Form 20035 für Wintersemester 2005/2006
// Bei Fehler wird FALSE zurückgegeben
 $startsemester = FALSE;
 // Select Personendaten des aktuellen Studenten
 $query = "SELECT Name,to_char(Studienbeginn,'yyyy') SBegJahr,to_char(Studienbeginn,'mm') SBegMonat
           FROM Studenten WHERE MatrNr = :Matrikelnummer";
 $result = oci_parse($dbcon, $query) or exit('Query parse failed: ' . oci_error());
 oci_bind_by_name($result, ":Matrikelnummer", $matrikelnummer, 5);
 oci_execute($result);

 // Drucke Name und Matrikelnummer des aktuellen Studenten
 $line = oci_fetch_array($result, OCI_ASSOC);
 if (!$line) {
  echo "<p>Keine gültige Matrikelnummer</p>";
  return FALSE;
 }
 else {
  echo "<h2>Notenausdruck für ", $line["NAME"], " (", $matrikelnummer, ")</h2><br>\n";
 }

 // Startsemester bestimmen
```

```
 if ($line["SBEGMONAT"]<6) {$startsemester = $line["SBEGJAHR"] * 10;}
 else {$startsemester = $line["SBEGJAHR"] * 10 + 5;}

 oci_free_statement($result); // beende Abfrage
 return $startsemester;
}

function druck_Notenliste($matrikelnummer, $startsemester, $dbcon) {
// Funktion druckt die Notenliste für den Studenten mit $matrikelnummer
// kein Rückgabewert
// Noten auslesen
 $query = 'SELECT Kurzname, Note, Versuch, Semester
           FROM nimmt_teil, Kurse
           WHERE nimmt_teil.KursNr = Kurse.KursNr AND MatrNr = :Matrikelnummer
           ORDER BY StdSem, Kurse.KursNr, Versuch';
 $result = oci_parse($dbcon, $query) or exit('Query parse failed: ' . oci_error());
 oci_bind_by_name($result, ":Matrikelnummer", $matrikelnummer, 5);
 oci_execute($result);

 // Tabelle mit Noten in HTML drucken
 echo "\n<table border=1>\n\t<tr>\n";
 // Drucke Spaltenüberschriften: "Fach" und Semester für 15 Semester ab Startsemester
 echo "\t<td><strong>Fach\\Semester</strong></td>";
 for ($i= $startsemester; $i < $startsemester + 75; $i = $i + 5) {
  // Wandle Semesternummern wie z. B. 20065 in lesbare Semesterangaben um: W 2006/2007
  if (($i % 10) == 0) { // Sommersemester
   $semester = $i / 10;
   echo "\n\t\t<td><strong>S $semester</strong></td>";
  }
  else { // Wintersemester
   $semester = ($i - 5) / 10;
   echo "\n\t\t<td><strong>W $semester", "/", ($semester % 10) + 1, "</strong></td>";
  }
 }
 echo "\n";

 //Noten drucken
 while ($line = oci_fetch_array($result, OCI_ASSOC)) {
   /* Dieser Programmabschnitt funktioniert nur, wenn es zu einem Versuch 2
      oder 3 auch einen Versuch 1 gibt; sonst wird eine falsche
         Tabellenstruktur ausgedruckt. */
   $note = $line[NOTE] / 10; // Note in Datenbank gespeichert als Note * 10
   if ($line[VERSUCH]==1) {
    echo "\t</tr>\n"; // beende alte Tabellenzeile
    echo "\t<tr>\n"; // beginne neue Tabellenzeile
       echo "\t\t<td>$line[KURZNAME]</td>\n\t\t";
    for ($i=$startsemester; $i < $line[SEMESTER]; $i = $i + 5) {echo "<td> </td>";}
   }
   else {
    for ($i=$i+5; $i < $line[SEMESTER]; $i = $i + 5) {echo "<td> </td>";}
   }
   echo "\n\t\t<td>$note ($line[VERSUCH])</td>\n";
 }
 echo "</tr>\n</table>\n";
 oci_free_statement($result); // beende Abfrage
}
?>
```

PHP-Anweisungen

- Tag für php-Code:
```
<?php
echo "code";
…
?>
```
- **Variable, Zuweisung:** $eineVariable = 5;
  $eineVariable = "ein String";
  Variablen sind nicht typgebunden.
- **Arrays:**
```
$anrede =  array ("Herr", "Frau");
echo $anrede[0];
```
- **Dictionary:**
```
$person = array ("vorname" => "Martin",  "nachname" => "Hulin", "email" =>
"hulin@hsw.de");
$person["beruf"] = "Prof.";
```
- **Ausgabe:**
```
echo "Vorname: ",    $person["vorname"];
print "Nachname: ";
print $person["nachname"];
```
- **Fallunterscheidung:**
```
if ($x < 7 || $x > 10) {$y = $x – 5;}
elseif ($x == 8) {$y = 2 * $x;}
else {echo "falscher Wert";}
```
- **while-Schleife:**
```
$a = 1; $b = 1;
while ($a <= 10) {
  $b = $a * $b;
  $a++;
}
```
- **for-Schleife:**
```
for ($a=0; $a <= 10; $a++) {$b = $a * $b;}
```
- **foreach-Schleife:**
```
$a = array (1, 5, 7, 9);
foreach ($a as $x) {$b = $b + $x;}
```
- **Funktionen**
```
function plus ($a, $b=1) {
  return ($a + $b);
}
echo plus(5, 7);
```
- **Datenbank-Zugriff für MySQL und Oracle**
  Hier ist nur eine kleine Auswahl möglicher Zugriffsfunktionen aufgelistet. Die komplette Funktionsübersicht finden Sie im Internet auf http://www.php.net/manual/en/

Funktion	MySQL	Oracle

Verbindung zum DB-Server	`$dbcon = mysql_connect(`DB-Server, User, Passwort`)`	`$dbcon = oci_connect(`User, Passwort, DB-Server)
Datenbank auswählen	`mysql_select_db(`DB-Name`)`	
Abfrage ausführen	`$result = mysql_query(`SQL-String`)`	`$result = oci_parse($dbcon,` SQL-String); `oci_execute($result)`
Cursor (Result-Set) durchlaufen	`$line = mysql_fetch_array($result, MYSQL_ASSOC)`	`$line = oci_fetch_array($result, OCI_ASSOC)`
Spalte addressieren	`$line[`Spalten-Name`]`	`$line[`Spalten-Name`]`
Verbindung schließen	`mysql_close($dbcon)`	`oci_close($dbcon)`

- **Datenbank-Abstraktions-Schichten**
  Ähnlich wie ODBC den Zugriff auf verschiedene DBMS erlaubt, gibt es auch bei PHP Datenbank-Abstraktions-Schichten, die für den Zugriff auf viele DBMS geeignet sind, z. B. PEAR MDB2. Eine solche Abstraktionsschicht sollte immer dann benutzt werden, wenn man in einer Anwendung auf DB-Systeme verschiedener Hersteller zugreift oder es möglich sein soll, die Anwendung auf ein anderes DBMS zu portieren.

- **Objektorientierte Erweiterungen**
  Ab PHP 4 und neu konzipiert in PHP 5 kann man in PHP auch objektorientiert programmieren. Dies empfielt sich nur bei größeren Web-Projekten.

- **Klassen**
```
class Person { // eine Klassendefinition
  public $name; // ein Attribut
  private $geburtstag; // ein privates Attribut

  public function alter() { // eine Methode
    $gebstr = strtotime($this->geburtstag);
    if ($gebstr = -1) {return "unbekannt";}
    else {
      $diff = time() - $gebstr;
      return floor($diff/(24*60*60*365));
    }

  public function __construct($name, $geburtstag) {
    $this->name = $name;
    $this->geburtstag = $geburtstag;
  }
}
```

- **Vererbung**
```
class Student extends Person {
  private static $counter = 0; // ein Klassen-Attribut

  public $matrikelnummer;

  public function __construct ($name, $geburtstag) {
    parent::__construct($name, $geburtstag); // Zugriff auf Oberklasse
    $this->matrikelnummer = self::$counter++; // erzeuge neue MatrNr
  }
}
```

- **Instanziierung**
```
$stud = new Student ("Huber, Anne", "10 Oct 1985");
echo $stud->name, " ist ", $stud->alter(), " Jahre alt."
```

5 Objektorientierte Datenbanken

5.1 Konzepte der OODB

Nachteile des relationalen Modells:

- Nur sinnvoll bei Entities mit vielen Elementen, sonst viel Aufwand bei Tabellenstruktur für wenig Daten → RDB wenig geeignet für CAD.
- Information über eine Entitiy durch Normalisierung oft über viele Tabellen verteilt: Daten eines Objekts müssen mühsam durch Join zusammengesetzt werden.
- Semantische Information über Zugehörigkeit und Zusammenhang von Entities ist nicht darstellbar. (z.B. Mitarbeiter gegliedert in die Entity-Sets Sekretär, Ingenieur, Manager)
- Komplexe, strukturierte Datentypen für Tabellenspalten sind nicht möglich.

Diese Nachteile sollen in OODBn vermieden werden.
OODBn wurden angeregt durch den Erfolg der OO-Programmierung.
Konzepte von OODBn bauen auf den Konzepten der objektorientierten Programmierung auf

Objekte:

- sind zusammengefasst in Klassen
- haben Eigenschaften
- haben systemweit eindeutigen Bezeichner
- haben Methoden.
- Vererbung von Klassen sowie Klassenhierarchien
- Abstrakte Datentypen → „information-hiding“
- Nachrichten an Objekte lösen Operationen (Methoden) aus.

Diese Konzepte wurden auch für OODB verwendet.

Ziele der OODB

- Reale Welt ist dargestellt als Vielfalt von miteinander in Beziehung stehenden Objekten.
- Klasseneinteilung der Objekte zur Strukturierung
- Untersuchung auf verschiedenen Abstraktionsebenen.
  z.B. Ort ist Attribut einer Adresse von Person,
  oder Klasse Ort mit eigenen Attributen.

Verwirklichung

- Durch Klassenbildung und Zusammenfassen ähnlicher Klassen zu Objektklassen, kann dann der Zusammenhang der Entities ausgedrückt werden. Außerdem können so auch Entity-Sets mit wenigen Elementen sinnvoll gespeichert werden (Overhead) !
- Abstrakte Datentypen vermeiden die Zerteilung von Daten einer Entity auf viele Tabellen

Beispiel: Person hat als Attribut eine Menge von Hobbies

<u>RDB:</u> *Person* (PNr, Name, GebDat, ...)
Hobby (PNr, Hobby_Name)

z.B.: Huber hat Hobbies: Fußball, Schach, Computerspiele
→ Hobbies müssen in vielen Zeilen der Tabelle „Hobbies“ gesucht werden.

OODB: Definiere den abstrakten Datentyp Menge mit Funktionen (Methoden): Neue_Menge, Füge_ein, Element, Vereinigung, ... Dann ergibt sich als Klassendefinition für *Person*

```
class       Person
properties  PNr       : integer;
            Name      : string;
            GebDat    : Datum;
            Hobbies   : Set (Hobby_typ);
operation   Neue_Person;
            Alter;
```

Durch das Geheimnisprinzip ist die Implementierung der Menge der Hobbys verborgen.
Daraus folgt:
- Interne Implementierung als eigene Tabellen möglich.
- Realisierung einer aufgesetzten OODB auf einer relationalen ist möglich.

Mit Abstrakten Datentypen (ADT) lassen sich auch komplexe, strukturierte Attribute aus benutzerdefinierten Typen realisieren.

Beispiel: Punkt = (x-, y-Koordinate)

```
class       Dreieck
properties  Ecke A: Punkt;
            Ecke B: Punkt;
            Ecke C: Punkt;
```

Aggregation: Zusammenfassen von Objekten zu neuen Objekten (Nicht zu verwechseln mit Klassenhierarchien).

Beispiel:

```
class       Fachhochschule
inherits    Hochschule
properties  Rektor          : Dozent
            Fachbereiche    : Set (Fachbereich)
                . . .
```

Die Properties Rektor und Fachbereiche, sind vollständige Objekte in einem Objekt der Klasse Fachhochschule (nicht nur Foreign Keys) wie im RDBM oder Pointer im NDBM).
Der Zugriff erfolgt ohne vorher programmiertem Join bzw. Navigation entlang von Pointern.
Der Zugriff kann so implementiert sein, aber das ist für den Benutzer verborgen.

Klassenbildung und Klassenhierarchien durch Generalisierung und Vererbung

Zusammenhänge zwischen Entities können ausgedrückt werden:
Beispiel:

```
class       Person
properties  Name        : string
            Adress      : String
            GebDat      : Datum
            Geschlecht  : (m,w)
operations  Neue_Person
            Alter ...
end
```

```
class        Student
inherits     Person
properties   MatrNr        : number (5)
             Studiengang   : char
             Studienbeginn : Datum
             nimmt_teil    : Set (Kurs)
operations   Studiensemester
end

class        Dozent
inherits     Person
properties ...
end;
```

DB - Integrität bei OODB durch

a.) Klassen und Vererbung

Beispiel:

```
class        Frau
inherits     Person
             ....
constraint   Geschlecht = 'w'
end;
```

b.) Angabe von Relation und Umkehrrelation

Beispiel:

```
class        Kurs
properties   KNr          : String
             KBez         : String
             ....
             Teilnehmer   : Set (Student)     inverse is Student.nimmt_teil
             Lehrer       : Dozent            inverse is Dozent.lehrt
end;
```

c.) Trigger wie bei RDB

5.2 Entwurf von OODB

Methodik

- Analyse der realen Welt, Erkennen der Objekte, Objektklassen und Attribute.
- Erkennen und Definieren der Funktionen (Methoden) für die Objekte.
- Schnittstellendefinition der Objekte
- Implementierung.

Anwendungsprogrammierung und Strukturentwurf laufen Hand in Hand.

Fragen: Welche Objektklassen sollen eingerichtet werden?
z.B. Person hat Hobbies ?

a) bei Hobby interessiert nur Name → Attribut bei Person
b) jedes Hobby hat genaue Beschreibung d.h. eigene Attribute wie Sportverein, Geräte... → eigene Klasse

Operationen gehören als Methoden zu den Klassen.
Daher müssen auch allgemeine Funktionen wie Einfügen, Löschen, und Ändern, bei jeder Klasse explizit definiert werden. Benutzerfreundliche OODB-Systeme werden dem Designer einen Satz häufig benutzter Methoden bei der Definition neuer Klassen zur Auswahl anbieten und automatisch zufügen.

5.3 *Sprachen, Abfragen*

- Kombination mit OO-Programmiersprache (C++, Smalltalk)
- Datenmanipulation durch Konstruktionsoperationen der Objektklassen z.B. Neue_Person
- Keine Standardsprache, wie SQL bei RDB
- Es gibt prozedurale, funktionale und SQL-ähnliche Abfragesprachen.

Beispiele in OQL:

a) Selektiere die Namen aller Studenten des Studiengangs AI im 7. oder höheren Semester:

```
SELECT  S.Name
FROM  S  IN  Student
WHERE  S.Studiengang = ´AI´ AND
   S.Studiensemester (S) > 6
```

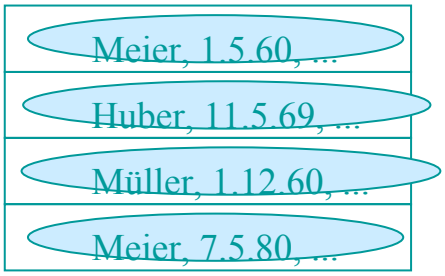
b) Selektiere die Namen der Dozenten, die den Studenten mit der Matr.Nr = ´1234´ unterrichten.:

```
SELECT  d.Name
FROM  Dozenten d, k IN d.Kurse, t IN k.Teilnehmer
WHERE  t.MatrNr  = ´1234´;
```

5.4 *Objekt-relationale Datenbanken: Beispiel mit Oracle 10g*

Objekt-Tabellen statt Tupel-Tabellen

Objekt-Tabelle Personen



Personen_typ:
Name,
Geburtsdatum,
...

Tupel-Tabelle Personen

Name	Geburtsdatum	...
Meier	1.5.60	
Huber	11.5.69	
Müller	1.12.60	
Meier	7.5.80	

Trennung von Strukturdefinition und Tabellendefinition als Objektmenge statt Tupelmenge

```
-- Ein eingebetteter Typ: Adresse
create type adress_typ as object
(strasse varchar2(50),
 hausnr  varchar2(5),
 plz char(5),
 ort varchar2(50));

-- Ein eingebetteter Typ: zusammengesetzter Name
create type name_typ as object
(titel varchar2(10),
 vorname varchar2(50),
 nachname varchar2(50),
 map member function namensort return varchar2);

create or replace type body name_typ as
map member function namensort return varchar2
is
 concatname varchar2(100);
begin
 concatname := nachname || vorname;
 return concatname;
end namensort;
end;

-- Ein mehrwertiges Attribut als Liste
create type telefonarray as varray(5) of varchar2(30);

-- Ein Typ mit Aggregation und mit Methoden
create type person_typ as object
(pname name_typ,
 adresse adress_typ,
 gebdat date,
 geschlecht char(1),
 telefonliste telefonarray,
 ref_vater ref person_typ,
 ref_mutter ref person_typ,
 member function alter_jahre return integer,
 pragma restrict_references (alter_jahre, wnps, wnds)) not final;

-- Student_Typ ist Subtyp von Person_Typ
create type student_typ under person_typ
(Studiengang char(3), Matrikelnr integer);
```

Aufgabe: Definiere den Subtyp Dozent_Typ von Person_Typ

_____

_____

_____

```
-- Definiere die Methode alter_jahre
```

```
create type body person_typ as
member function alter_jahre return integer
is
 jahrg integer; jahrj integer;
 monatg integer; monatj integer;
 tagg integer; tagj integer;
begin
 jahrg := to_char(gebdat, 'yyyy');
 jahrj := to_char(sysdate, 'yyyy');
 monatg := to_char(gebdat, 'mm');
 monatj := to_char(sysdate, 'mm');
 tagg := to_char(gebdat, 'dd');
 tagj := to_char(sysdate, 'dd');
 if monatj > monatg or (monatj = monatg)
 then return jahrj - jahrg;
 else return jahrj - jahrg - 1;
 end if;
end alter_jahre;
end;
```

Aufgabe: Definiere den Typ Kurs_Typ mit Referenz auf einen Dozenten

```
-- Erzeuge einen Behälter (Extent) für den Personen-Typ
create table personen of person_typ;

-- Füge zwei Personen-Objekte zur Personen-Tabelle hinzu
insert into personen values
(person_typ(name_typ('Dr.', 'Hans', 'Huber'),
 adress_typ ('Panoramastrasse', '88', '88888', 'Buxtehude'),
 '11.11.1991', 'm',
 telefonarray('0555/555555', '0555/555-9555', '0171/987654'), NULL, NULL));
insert into personen values
(person_typ(name_typ(NULL, 'Heinrich','Huber'),
NULL,
 '01.01.1910', 'm',
NULL, NULL, NULL));

-- Zeige beide Datensätze an
select p.pname, p.adresse, p.gebdat, p.geschlecht, p.telefonliste from personen p;

-- Heinrich Huber ist der Vater von Hans
update personen p
```

```
set ref_vater =
  (select ref(v) from personen v where v.pname.vorname = 'Heinrich' and v.pname.nachname =
'Huber')
where p.pname.vorname = 'Hans' and p.pname.nachname = 'Huber';

-- Suche alle Personen mit ihrem Vater
select p.pname, p.adresse, p.gebdat, p.geschlecht, p.telefonliste, p.ref_vater.pname from personen p;

-- Selektiere eine Methode
select p.pname, p.alter_jahre() from personen p;
```

Aufgabe: Definiere eine Tabelle für Kurse

```
-- Füge eine Studentin in die Personentabelle ein
declare s student_typ;
begin
 s := student_typ(name_typ(NULL, 'Hilde', 'Huber'),
 adress_typ ('Panoramastrasse', '88', '88888', 'Buxtehude'),
 '11.11.1988', 'w',
 telefonarray('0555/555555', '0171/987654'), NULL, NULL, 'AI', 1234);
 insert into Personen values s;
end;

-- Suche nur die Studenten in der Personentabelle
select value(s) from Personen s where value(s) IS OF (Student_Typ);
```

Aufgabe: Füge einen Dozenten in Personen ein

Aufgabe: Erzeuge einen Kurs mit diesem Dozenten

Aufgabe: Suche die Bezeichnungen aller Kurse des Dozenten Max Meier

6 Interne Funktionsweise von Datenbankmanagementsystemen

- Datenintegrität
- Sicherheit in Datenbanken
- Transaktionen und Fehlerbehandlung
- Nebenläufige Verarbeitung
- Abfrageoptimierung
- Verteilte Datenbanken
- DB-Katalog
- Physikalische Datenspeicherung
- Zeitaspekte bei DBn
- Zugriff auf kommerzielle DBn
- Abrechnung in DBn
- Speichern von Multimedia-Daten in Datenbanken

6.1 Sicherheit in Datenbanken

- Information muss geschützt werden vor unerlaubtem Zugriff (lesend und schreibend)
- Zusammenarbeit mit Betriebssystem
- Verschlüsselung von Daten (in / außerhalb DBMS)
- Unterschiedliche Zugriffsrechte
- Matrix von Zugriffsrechten (R = read, W = write, Programmausführungsrecht Yes/No)

	Tab1	Tab2	Tab3	. . .	Prog1	Prog2	. . .
Ben1	RW	-	-		Y	Y	
Ben2	R	R	-		Y	N	
Ben3	R	-	-		N	N	
.							
.							

- Alternative: Benutzer Profiles
  Ben1: Tab1, Prog1, Prog2
  Ben2: Tab1(R), Tab2(R), Prog1
  Ben3: Tab1(R)
  . . .
- Oder: Objekt Profiles
  Tab1: Ben1, Ben2(R), Ben3(R)
  Tab2: Ben2(R)
  . . .
- Oder Einträge in Systemtabelle für Benutzerrechte:

Benutzer	**Objekt**	**Recht**
Huber	Arena	Select
Huber	Arena	Insert
...		

- Zuständige SQL-Anweisung: GRANT Select ON Arena TO Huber;
- Systemprivilegien werden ebenfalls durch GRANT vergeben.
- Rollenkonzept

6.2 Transaktionen und Fehlerbehandlung

- In einem Programm müssen oft alle Aktionen zusammen durchgeführt werden.
- Beispiel: Speichern einer Person und Ihrer Kommunikationsadressen
- Person Max Müller, geboren am 26.3.1967, ledig hat seinen Hauptwohnsitz in 88212 Ravensburg, Karlstraße 15
  und seinen Nebenwohnsitz in 9315 Kleinhofen, Parkstraße 7

- Die zusammengefassten Teilaktionen heißen **Transaktion**.
- Definition Transaktion:
  - Logische Zusammenfassung von Aktionen
  - Konsistenter Zustand der DB vorher und nachher
- **Ablauf:**

Start Transaktion (automatisch)
Aktion 1:
insert into Person values
(444, 'Max', 'Müller', '26.3.1967', ledig);
Aktion 2:
insert into Adressen values
(444, 88212, 'Ravensburg', 'Karlstraße 15', 'H');
Aktion 3:
insert into Adressen values
(444, 9315 'Kleinhofen', 'Parkstraße 7', 'N');
Ende Transaktion: Commit (Bestätigung)

- Bei Fehler (Systemfehler, Transaktionsfehler) **Rollback** in vorigen Zustand.
- Dazu **Log-File**: Es enthält für jede Transaktion
- Transaktion-Identifier, alte Werte, neue Werte, committed, . . .
- Ablauf Transaktion intern im DB-Cache

DB-Block Puffer	Rollback-Segm. Puffer	Redo-Log Puffer TransID

- Bei Datenverlust (z. B. Disc-Crash) muss auf eine alte Sicherungskopie zurückgegriffen werden. Diese wird mittels Log-File auf den aktuellen Stand gebracht: **Rollforward**
- Dazu regelmäßige Sicherungskopien
- Andere Methoden zur Fehlerbehebung: **Shadow-Datenbank**
- Effizienter: Shadow-DB produktiv einsetzen: Real Application Cluster (RAC)

6.3 Nebenläufige Verarbeitung (concurrent processing)

Situation: Mehrere Benutzer greifen unabhängig voneinander, gleichzeitig auf die DB zu, evtl. auf die gleichen Daten.
Beispiel : Zwei berechtigte Personen heben gleichzeitig an verschiedenen Filialen Geld von ihrem Konto ab:

Filiale 1				
Filiale 2				
				Zeit

Lösung: Daten werden vor der Änderung durch eine Transaktion für den Zugriff von anderen Transaktionen gesperrt : **Locking**

Granularität eines Locks: Umfang der Daten, die gesperrt werden: ganze DB, einzelne Tabellen, Zeilen oder Spalten, einzelne Daten.

exklusiver Lock: keine andere Transaktion darf auf gesperrte Daten zugreifen, weder lesend noch schreibend:

shared Lock : keine andere Transaktion darf auf die gesperrten Daten schreibend zugreifen.

Gefahr bei Locking : Deadlock

Beispiel: 3 Transaktionen TA1, TA2 und TA3
TA1 benötigt Zugriff auf Daten D1 und D2
TA2 benötigt Zugriff auf Daten D2 und D3
TA3 benötigt Zugriff auf Daten D3 und D1

zeitlicher Ablauf:

TA1						
TA2						
TA3						

Deadlock: alle Transaktionen warten

Lösung von Deadlocks :

1. Vorsorge: Alle Transaktionen müssen ihre Betriebsmittel vorher anfordern
2. Deadlock Erkennung : Suche nach Zyklen in einem Wartegraph.

Lösen durch Rollback einer der beteiligten Transaktionen. Diese wird später erneut gestartet.

TA1 TA2

TA3

Wartegraf für obiges Beispiel

6.4 Datenbankintegrität (siehe Power-Point-Präsentation)

- Forderung: Konsistente Daten in DB
- Auch bei vielen gleichzeitigen Nutzern
- Auch bei Fehlersituationen
- Datenbank konsistent zu halten, ist Aufgabe des Datenbank-Management-Systems
- DBMS braucht Zusatzinformation
- Verschiedene Methoden
  - Deklarative Integritätsmethode
  - Prozedurale Integritätsmethode beim Client
  - Prozedurale Integritätsmethode in DB
  - Aktive Tabellenfilter

6.4.1 Deklarative Integritätsmethode: Constraints

- Einsatz: Überprüfung von Standardfällen
- Parametrisiert
- Bei Tabellendefinition oder später
- Constraints können ein- oder ausgeschaltet werden (enable or disable)
- Effizient implementiert aber natürlch trotzdem zusätzlicher Zeitbedarf, um die Regeln zu prüfen
- Inkonsistente Datensätze können automatisch in Fehlertabelle verschoben werden

Arten von Constraints:
Primary Key: siehe Kapitel 3, SQL, create table
Unique: jeder Datensatz hat eindeutigen Wert bei dem Unique-Attribut
Not Null: jeder Datensatz muss bei diesem Attribut einen Eintrag haben.
Primary Key = Unique + Not Null
Check Constraint: Überprüfen bestimmter Eigenschaften, z. B.

Foreign Key: siehe Kapitel 3, SQL, create table;
referentielle Integritätsaktion: **on delete cascade**
Default: wenn bei Insert in dieser Spalte nichts eingetragen wird, wird der default-Wert gesetzt.

Ausführungszeitpunkt

- sofort (immediate) bei Insert, Update, Delete
- verzögert (deferred) am Ende der Transaktion

6.4.2 Prozedurale Integritätsmethode beim Client

- Client-Programm überprüft Eingabe vor Senden an Server
- Vorteile:
  - flexibel
  - benutzerfreundlich
- Nachteile
- unsicher, da Schlupflöcher
  - Überprüfung in vielen Client-Programmen nötig ⇒ hoher Programmieraufwand
  - andere Daten aus DB sind nicht direkt verfügbar, sondern müssen von DB geholt werden

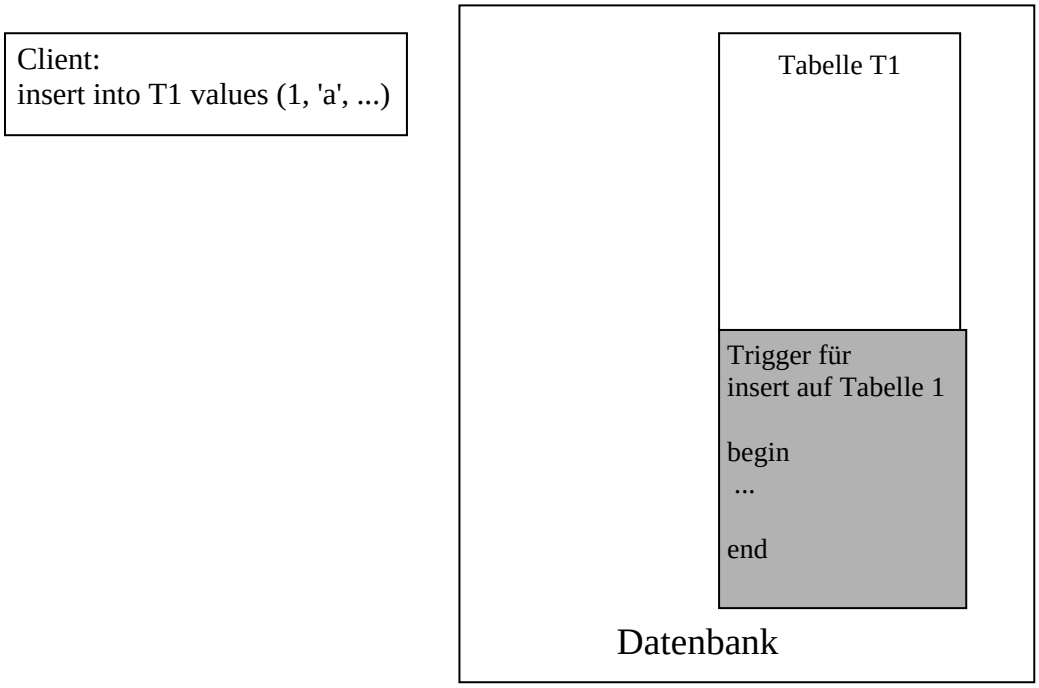
6.4.3 Prozedurale Integritätsmethode in DB: Trigger

- Benutzerdefinierter Zusatz zu DBMS
- Selbstgeschriebene Prozeduren
- Ereignisgesteuerte Auslösung
- Läuft komplett auf Server

Vorteile
- flexibel
- wirkt für alle Zugriffsmöglichkeiten auf DB

Nachteile
- Erkennung von Fehleingaben erst am Server
- evtl. lange Laufzeit

Abbildung 6-1: Ablauf eines Triggers



create or replace trigger <trigger-name>	Trigger-Name
before | after | instead of	Auslösezeitpunkt
insert or update of <Spalte1>, <Spalte2>, ... or delete on <Tabellenname>	Trigger-Ereignis
for each row	Datensatz- oder Befehls-Trigger
when <Bedingung>	Trigger-Bedingung
PL-SQL	Trigger-Rumpf mit eigentlichem Trigger-Programm

Abbildung 6-2: Aufbau eines Triggers

Trigger-Typen

Auslösezeitpunkt **before**

- verhindert Eintrag fehlerhafter Datensätze
- Trigger wird vor dem auslösenden Ereignis ausgeführt

Auslösezeitpunkt **after**

- korrigiert redundante Information
- Trigger wird nach dem auslösenden Ereignis ausgeführt

instead of Trigger

- Damit werden insert-, update- und delete-Operationen auf Views möglich
- Trigger wird nicht vor oder nach einer insert-, update- und delete-Operation ausgeführt, sondern an Stelle dieser Operation (instead of)

12 Kombinationen im Trigger-Kopf möglich

	Zeitpunkt		Trigger-Typ	
	before	after	Befehl	Datensatz
insert	x	x	x	x
update	x	x	x	x
delete	x	x	x	x

Zugriff auf auslösenden Datensatz

- Zugriff auf aktuellenDatensatz mit :old oder :new,
  z. B. :old.Gehalt
- :old bezeichnet den Datensatz
  - vor der Änderung bei update
  - vor dem Löschen bei delete
- :new bezeichnet den Datensatz
  - nach der Änderung bei update
  - nach dem Einfügen bei insert

Spezielle Bedingungen

- Problem: Trigger wird bei mehreren Ereignissen ausgelöst
- Aber: Manche Aktionen nur bei bestimmten Ereignissen
- dann Unterscheidung im Trigger-Rumpf durch
  - if inserting then
  - if deleting then
  - if updating then
  - if updating ('<Spalte>') then

Abbruch des Triggers

Trigger bricht ab bei

- Datenbank-Fehler, z. B. constraint-Verletzung
- benutzerdefinierten Fehlern

Explizites Setzen von Fehlern durch: **raise_application_error(<Fehler-Nr>, <Fehler-Text>)**
Im Fehlerfall wird Transaktion mit Rollback rückgängig gemacht.

Beispiel 1: Trigger verhindert Fehleingabe

- Für jeden Professor ist in der Spalte *Fakultät* eingetragen, zu welcher Fakultät er gehört
- Es gibt eine Tabelle **Gremien**, in der Gremien wie Fakultätsräte, Prüfungsausschüsse usw. gespeichert werden.
- Auch die Tabelle Gremien hat ein Feld *Fakultät*. Bei Gremien, die nicht zu einer Fakultät gehören, ist "HS" eingetragen.

Name	Tel	...	Fakultät	PersNr
Löhmann	9590		E	1234
Keller	9734		E	5678
Krönes	9583		T	9012
Plewa	9441		S	3456
...				

Gremium	Fakultät	Aufgabe	…	GNr
Fakultätsrat	E	Entsch…		11
Fakultätsrat	T	Entsch…		222
Senat	HS	Das …		1
…				

- Professoren sind Mitglieder von Gremien. Dies ist in einer Verbindungstabelle **Gremienmitglieder** mit den Fremdschlüsseln PersNr und GNr eingetragen.
- Bei Gremien der Fakultät dürfen nur Professoren dieser Fakultät Mitglied sein.
- Überprüfung durch einen before-Trigger bei
- insert in die Tabelle Gremienmitglieder

```
create or replace trigger
before after
insert or
update of
delete
on
for each row
declare
```

```
begin
    if
```

```
end;
```

Beispiel 2: Trigger berichtigt redundante Daten

- In der Tabelle Studiengaenge wird die Anzahl der Studenten im Feld AnzStud gespeichert.
- Redundanz zu Stg bei Tabelle Studenten
- AnzStud muss korrigiert werden bei
  - Einfügen eines neuen Studenten (Immatrikulation)
  - Löschen eines Studenten (Exmatrikulation)
  - Änderung des Studiengangs eines Studenten

```
create or replace trigger korr_AnzStud
before after
insert or
update of
delete
on
for each row
declare
```

```
begin
    if inserting or updating then
```

```
end;
```

Beispiel 3: komplexer Trigger mit Cursor

- Neuer Datensatz in Tabelle "nimmt_teil"
- Überprüfe, ob der Student in diesem Semester bereits angemeldet ist.
- Überprüfe, ob dieser Student diese Prüfung bereits bestanden hat.
- Der neue Versuch darf maximal 2 Semester nach dem letzten Versuch liegen

```
CREATE OR REPLACE
TRIGGER NEUER_VERSUCH
 BEFORE INSERT ON NIMMT_TEIL
FOR EACH ROW

DECLARE
```

```
BEGIN
```

```
END;
```

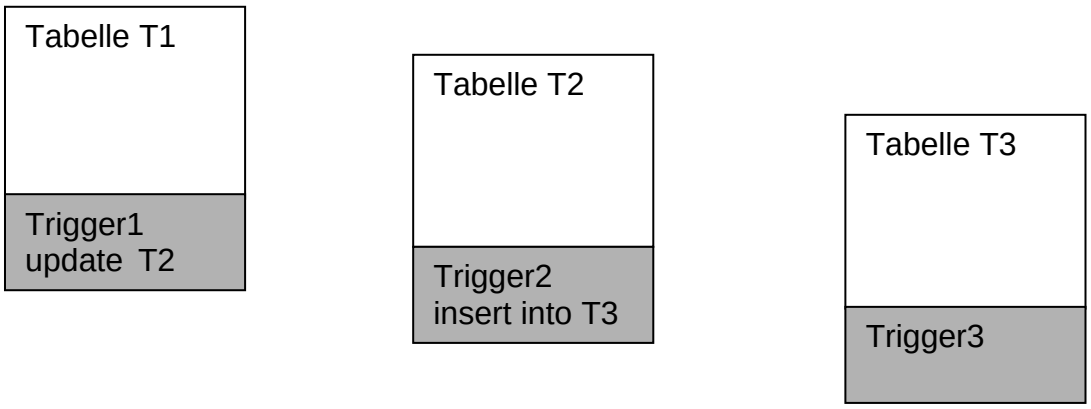
Zugriff auf die eigene Tabelle

- Zugriff auf eigene Tabelle verboten bei FOR EACH ROW - Trigger
- Beispiel:

```
create trigger falsch after insert on Studenten for each row
begin
   update Studiengaenge set AnzStud =
        (select count(MatrikelNr) from Studenten
         where Stg = :new.Stg)
end;
```

- Lösung 1: Dummy-Tabelle und zwei Trigger
- Lösung 2: FOR EACH STATEMENT - Trigger

Kaskadierende Trigger



INSTEAD OF Trigger

- erlauben schreibenden Zugriff auf Views
- Aktion auf View (UPDATE, INSERT oder DELETE)
  löst Trigger aus
  startet Aktionen auf Basistabellen
- Beispiel 4: Einfügen in folgende View "Studentinnen"

```
CREATE VIEW Studentinnen AS
SELECT Name, Adresse, Geb_Dat, MatrNr, Studienbeginn,
Studiengang
FROM Studenten
WHERE Geschlecht = 'w';
```

- Bei INSERT in diese VIEW muss das Geschlecht automatisch auf 'w' gesetzt werden.
- Trigger:

```
CREATE OR REPLACE
TRIGGER NEUE_Studentin
 INSTEAD OF INSERT ON Studentinnen
FOR EACH ROW
BEGIN









END;
```

Aktive Tabellenfilter

- Erweiterung des VIEW-Konzepts
- VIEW-Abfrage wird dynamisch ergänzt
- Ergänzung hängt vom Benutzer ab
- Ergänzung wird von benutzerdefinierte Funktion ermittelt
- Beispiel: VIEW "MeineNotenliste"

- Eingabe des Studenten:

- Ausgeführte Anweisung

6.5 Abfrageoptimierung

Netzwerk-DB: Optimierung durch Programmierer mittels DB - Navigation.

Relationale DB: Ein SQL - Befehl kann in vielen verschiedenen Arten verarbeitet werden. Der Abfrage -Übersetzer oder -Interpreter soll die optimale Bearbeitung wählen.

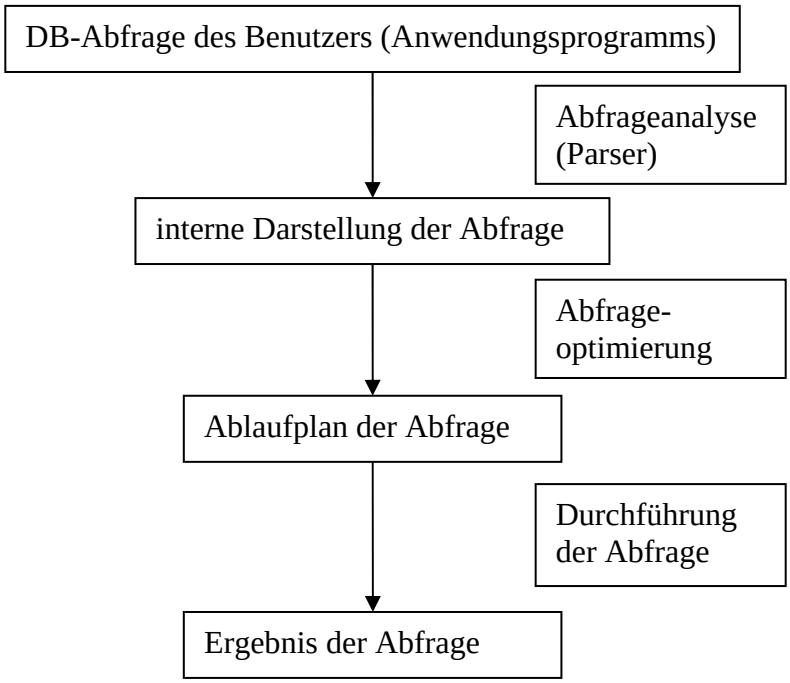


Abbildung 6-3: Bearbeitungsschritte einer Abfrage

Es gibt verschiedene Verfahren für Optimierer

aber: Abfrageoptimierung ist **Np-vollständig**

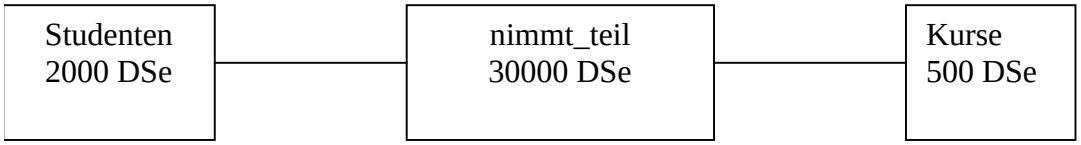
und die Zeit für die Optimierung muß zur Abfragezeit hinzugerechnet werden.

Verfahren:

1. - Erstelle irgendeinen korrekten Ablaufplan.
   - Wende Umstellungsregeln und Heuristiken darauf an, z.B.: „Erst Selektion, dann Join ist bessere Reihenfolge“.
   - Dadurch wird ein besserer Ablaufplan erzeugt.
2. - Erstelle mehrere mögliche Ablaufpläne.
   - Bewerte die Ablaufpläne.
   - Verwende Schätzformeln für die Zeitdauer der Grundoperationen (Selektion, Join) mit den Tabellengrößen als Parametern.
   - Der Ablaufplan mit dem geringsten geschätzten Zeitaufwand wird verwendet.

Andere Verfahren, z.B. neuronale Netze oder genetische Algorithmen.

Beispiel für verschiedene Ablaufpläne



```
SELECT Name, Bezeichnung
FROM Studenten s, nimmt_teil nt, Kurse k
WHERE
```

Plan 1

Für jeden Studenten
suche die jeweils passenden nt-Datensätze
und für alle passenden nt-DSe
suche den zugehörigen eindeutigen Kurs

Plan 2

Für jeden Datensatz in nt
suche den passenden DS in Studenten
und suche den passenden DS in Kurse

Der Optimierer legt aber keine eigenen Strukturen an z. B. Indices, das muss der Datenbankadministrator entscheiden

Beispiel Join :

Methode a) geschachtelte Schleifen; besser bei vielen Änderungen in den Tabellen aber seltenem Join

Für alle Zeilen von Tabelle T1
Für alle Zeilen von T2
Wenn T1.C = T2.C dann ...

Methode b) Sortiere erst T1 und T2 bezüglich C (Index auf C), dann führe Join durch; besser bei häufigem Join und wenig Änderungen

```
SQL> alter session set events '10053 trace name context forever';

Session altered.

SQL> select ename, dname
  2  from emp e , dept d
  3  where e.deptno=d.deptno
  4
```

```
Q*** SESSION ID:(19.9772) 2000.04.26.09.59.27.217
UERY
select ename, dname
from emp e , dept d
where e.deptno=d.deptno
****************************************
PARAMETERS USED BY THE OPTIMIZER
********************************
OPTIMIZER_PERCENT_PARALLEL = 0
OPTIMIZER_MODE/GOAL = Choose
HASH_AREA_SIZE = 131072
HASH_JOIN_ENABLED = TRUE
HASH_MULTIBLOCK_IO_COUNT = 0
OPTIMIZER_SEARCH_LIMIT = 5
SORT_AREA_SIZE = 131072
PARTITION_VIEW_ENABLED = FALSE
FAST_FULL_SCAN_ENABLED = TRUE
DB_FILE_MULTIBLOCK_READ_COUNT = 8
ALWAYS_STAR_TRANSFORMATION = FALSE
****************************************
BASE STATISTICAL INFORMATION
***********************
Table stats    Table: DEPT   Alias:  D
  TOTAL :: CDN: 4  NBLKS:  1  TABLE_SCAN_CST: 1  AVG_ROW_LEN:  23
-- Index stats
  INDEX#: 1984  COL#: 1
    TOTAL :: LVLS: 0   #LB: 1  #DK: 4  LB/K: 1  DB/K: 1  CLUF: 1
***********************
Table stats    Table: EMP   Alias:  E
  TOTAL :: CDN: 85600  NBLKS:  559  TABLE_SCAN_CST: 85  AVG_ROW_LEN:  14
-- Index stats
  INDEX#: 1986  COL#: 1
    TOTAL :: LVLS: 0   #LB: 1  #DK: 14  LB/K: 1  DB/K: 1  CLUF: 1
  INDEX#: 5980  COL#: 6
    TOTAL :: LVLS: 1   #LB: 28  #DK: 8  LB/K: 3  DB/K: 160  CLUF: 1287
****************************************
SINGLE TABLE ACCESS PATH
  TABLE: EMP     ORIG CDN: 85600  CMPTD CDN: 85600
  Access path: tsc  Resc:  85  Resp:  85
  BEST_CST: 85.00  PATH: 2  Degree:  1
****************************************
SINGLE TABLE ACCESS PATH
  TABLE: DEPT     ORIG CDN: 4  CMPTD CDN: 4
  Access path: tsc  Resc:  1  Resp:  1
  BEST_CST: 1.00  PATH: 2  Degree:  1
****************************************
OPTIMIZER STATISTICS AND COMPUTATIONS
****************************************
GENERAL PLANS
***********************
```

```
Join order[1]: DEPT [ D] EMP [ E]
Now joining: EMP *******
NL Join
  Outer table: cost: 1  cdn: 4  rcz: 22  resp:  1
  Inner table: EMP
    Access path: tsc  Resc: 85
    Join resc:  341  Resp:  341
Column:    DEPTNO  Col#: 8     Table: EMP   Alias:  E
    NDV: 3        NULLS: 5270     DENS: 3.3333e-001 LO:  10  HI: 30
Column:    DEPTNO  Col#: 1     Table: DEPT   Alias:  D
    NDV: 4        NULLS: 0        DENS: 2.5000e-001 LO:  10  HI: 40
Join cardinality:  80330 = outer (4) * inner (85600) * sel (2.3461e-001)  [flag=0]
  Best NL cost: 341  resp: 341
SM Join
  Outer table:
    resc: 1  cdn: 4  rcz: 22  deg: 1  resp: 1
  Inner table: EMP
    resc: 85  cdn: 85600  rcz: 14  deg:  1  resp: 85
    SORT resource     Sort statistics
      Sort width:          7 Area size:      44544   Degree: 1
      Blocks to Sort:      1 Row size:          35 Rows:        4
      Initial runs:        1 Merge passes:       1 Cost / pass:        3
      Total sort cost: 2
    SORT resource     Sort statistics
      Sort width:          7 Area size:      44544   Degree: 1
      Blocks to Sort:   1100 Row size:          26 Rows:     85600
      Initial runs:       50 Merge passes:       3 Cost / pass:     1416
      Total sort cost: 2674
  Merge join  Cost:  2762  Resp:  2762
SM Join (with index on outer)
  Access path: index (no sta/stp keys)
      INDEX#: 1984  TABLE: DEPT
      CST: 2  IXSEL:  1.0000e+000  TBSEL:  1.0000e+000
  Outer table:
    resc: 2  cdn: 4  rcz: 22  deg: 1  resp: 2
  Inner table: EMP
    resc: 85  cdn: 85600  rcz: 14  deg:  1  resp: 85
    SORT resource     Sort statistics
      Sort width:          7 Area size:      44544   Degree: 1
      Blocks to Sort:   1100 Row size:          26 Rows:     85600
      Initial runs:       50 Merge passes:       3 Cost / pass:     1416
      Total sort cost: 2674
  Merge join  Cost:  2761  Resp:  2761
HA Join
  Outer table:
    resc: 1  cdn: 4  rcz: 22  deg: 1  resp: 1
  Inner table: EMP
    resc: 85  cdn: 85600  rcz: 14  deg:  1  resp: 85
  Hash join one ptn:  20   Deg:  1
      hash_area:  64  buildfrag:  65  probefrag:  1087            ppasses:   2
  Hash join   Resc: 106   Resp: 106
Join result: cost: 106  cdn: 80330  rcz: 36
Best so far: TABLE#: 0  CST:        1  CDN:        4  BYTES:       88
Best so far: TABLE#: 1  CST:      106  CDN:    80330  BYTES:  2891880
***********************
Join order[2]: EMP [ E] DEPT [ D]
Now joining: DEPT *******
NL Join
  Outer table: cost: 85  cdn: 85600  rcz: 14  resp:  85
  Inner table: DEPT
.
.
```

Abbildung 6-4: Protokoll des Oracle-Optimierers

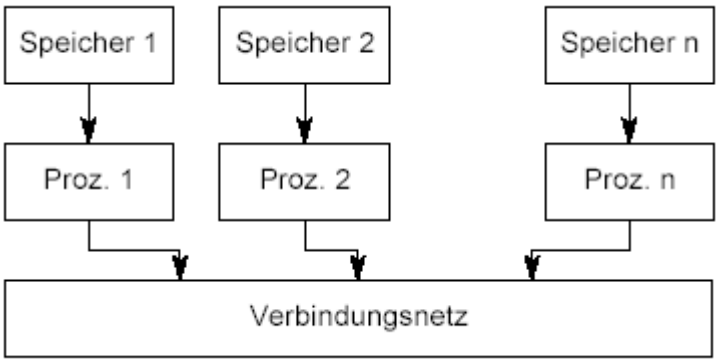
6.6 Verteilte Datenbanken

Anwendung 1 :

Mehrere Computer an verschiedenen Standorten, vernetzt. Die DB wird auf die Standorte verteilt, wenn überwiegend lokale Information genutzt wird.

Anwendung 2 :

Parallelrechner mit verteiltem Speicher: Die Datenbank wird auf die einzelnen Prozessoren mit ihren Speichern verteilt.



Vorteile :

- Parallelverarbeitung → Zeitgewinn
- Fehlertoleranz: Bei Ausfall eines Knotens können die anderen weiterarbeiten.
- Der Flaschenhals eines zentralen Servers wird vermieden.
- Kostenersparnis

Probleme :

1. Kommunikation zwischen verschiedenen Prozessoren ist relativ gesehen extrem zeitaufwendig.
2. Transaktionsmanagement bei verteilten Transaktionen.
3. Globaler Deadlock bei verteiltem Locking.
4. Konsistenz von Kopien.

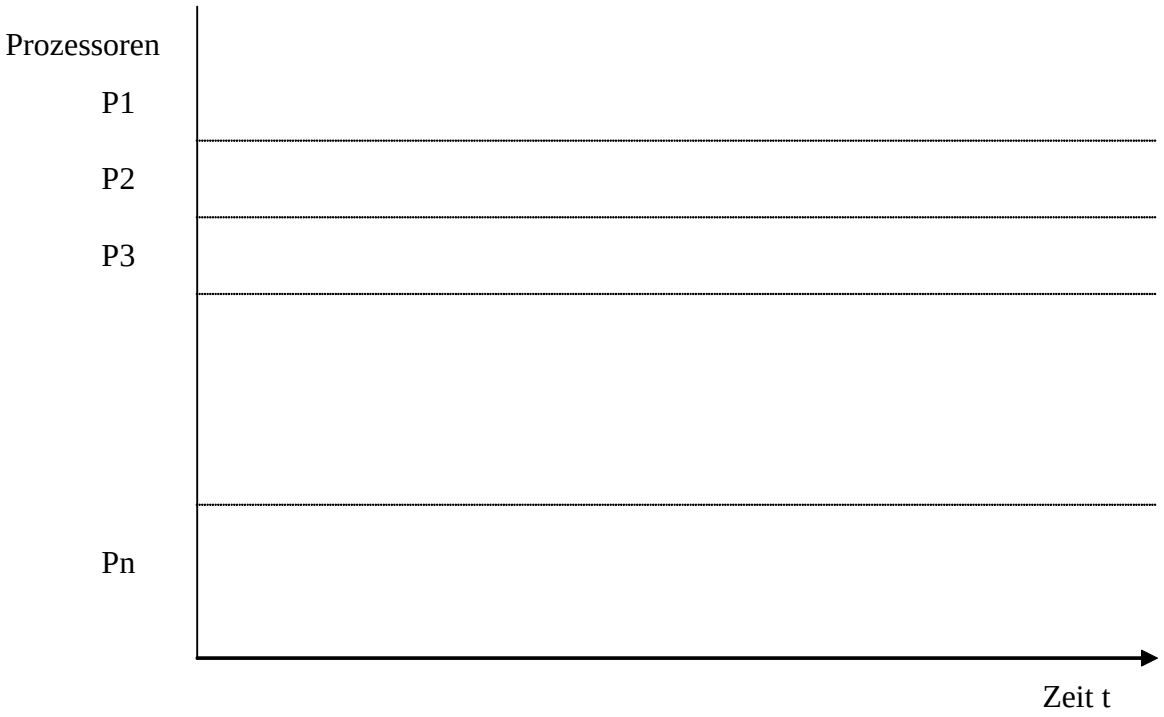
Problemlösungen :

zu 1.)

a.) optimale Verteilung der Daten auf die Prozessoren.

b.) Daten auf verschiedene Rechner kopieren (transparent für den Benutzer)

zu 2.) Zwei-Phasen-Commit



zu 4. a.) Bei Änderung des Originals auch alle Kopien ändern.
b.) Kopien für ungültig erklären.

6.7 Temporale Datenbanken

Nicht nur der aktuelle Datensatz wird gespeichert, sondern Historie
Beispiele

- Bankkonten
- Reservierungssysteme
- Patientendatenbanken im Gesundheitswesen

Granularität der Zeit: Chronon

- Mikrosekunden … Jahr
- Ereignisse innerhalb eines Chronons gleichzeitig

6.7.1 Zeitdarstellung, Kalender, Zeitdimensionen

Zeit braucht Bezugspunkt: Kalender
Verschiedene Kalender üblich
Umrechnung nötig
Zeitpunkte und Zeitdauer
Zeitsynchronisation mit realer Welt

- Datenbank mit Gültigkeitszeit
- Transaktionszeit-Datenbank
- Bitemporale Datenbank

6.7.2 Beispiele (siehe Vorlesung)

FH-Datenbank

- Zeitpunkte
- Datenbank mit Gültigkeitszeit

Mitarbeiter in Abteilung

- Zeitabschnitte
- Datenbank mit Gültigkeitszeit

Probleme	Lösungen
Abfragen werden komplizierter, z. B. Welche Mitarbeiter hat die Abteilung F&E?	Aktuelle Datensätze in extra Tabelle
Datenmenge wächst	Aktuelle Datensätze in extra Tabelle
Zusammengesetzter Schlüssel nötig	Surrogatschlüssel, RowID

6.7.3 Weitere Aspekte

Einfügen und Löschen
Datentyp: Zeitreihen
Objektorientierte, temporale Datenbanken
TSQL2: temporale Abfragesprache

6.8 Physikalische Speicherorganisation (siehe eLearning-Modul in MOODLE)

Bitte eignen Sie sich diesen Abschnitt selbständig mit den Lernmaterialien im Kurs Datenbanksysteme auf dem Lernsystem MOODLE an. Wenn dabei Fragen auftauchen, stellen Sie sie in der Vorlesung, schreiben einen Beitrag im Forum oder schicken mir eine E-Mail.